

本科毕业设计（论文）

基于 LayaAir 与 ECS 的 Roguelite 生存战斗游戏 设计与实现

学生姓名：（请填写姓名）

学 号：（请填写学号）

专业班级： 计算机科学与技术 （请填写班级）

指导教师：（请填写指导教师）

2026 年 6 月

学位论文原创性声明

本人所提交的学位论文《基于 LayaAir 与 ECS 的 Roguelite 生存战斗游戏设计与实现》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。本声明的法律后果由本人承担。

论文作者(签名):_____ 指导教师确认(签名):_____

年 月 日

年 月 日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

保密的学位论文在_____年解密后适用本授权书。

论文作者（签名）：_____ 指导教师（签名）：_____

年 月 日 年 月 日

摘 要

随着 HTML5 与 WebGL 技术的成熟，浏览器端已能够承载中等复杂度的实时动作游戏。Roguelite（轻 Roguelike）品类以单局随机性、局外元进度与构筑深度为核心乐趣，对架构的可扩展性、数据驱动能力以及运行时性能提出了较高要求。本文以毕业设计项目 *Survivor And Fight* 为对象，在 LayaAir 3.x 引擎与 TypeScript 技术栈上，设计并实现了一套面向 2D 俯视角生存战斗的完整游戏系统。

系统采用“类 Unity DOTS 思想”的轻量级实体组件系统（ECS）作为玩法内核，将位置、速度、属性、技能、操控等数据与逻辑解耦为组件与系统，并通过分组调度与命名筛选器支撑玩家、怪物及可操控实体的批量更新。战斗层实现了配表驱动的技能与效果执行链，支持子弹发射、穿透、分裂、连锁等组合效果；子弹与怪物均通过对象池减少实例化开销；在实体规模达到阈值时，战斗快照可通过 Web Worker 异步重算，主线程仅负责采集与应用结果，从而缓解帧时间尖峰。元游戏层提供开始界面、选关与三大关爬塔式跑图（有向无环图层、节点类型包括篝火、战斗、精英、商店等），局内支持经验升级、随机奖励与 Tab 键打开的技能装配面板（MVC + UI 栈）。工程上采用 OpenSpec 规格驱动开发，将 ECS 核心、玩法层、UI 栈、子弹与血量、怪物波次等能力拆分为可验证的变更提案，保证设计与实现的一致性。

本文从需求分析、总体架构、模块详细设计、性能优化与测试验证等方面展开论述，并结合模板要求对系统的社会影响、绿色节能与项目管理应用进行了讨论。全文共八章：绪论阐明背景与论文结构；第二章综述 LayaAir、ECS、Roguelite、数据驱动与 H5 性能等相关技术；第三章给出功能与非功能需求及用例；第四章描述分层架构、ECS 调度与数据流；第五章按模块详解 ECS、技能、子弹、怪物、元游戏、跑图、UI 与配表实现，并给出典型场景剖析与源码索引；第六章集中讨论对象池、Web Worker、碰撞与配置等优化手段；第七章说明测试策略、用例与性能实验框架；第八章总结工作并展望法杖系统、archetype 存储等方向。

测试表明，在目标平台上系统可稳定运行，对象池与 Worker 卸载对帧率稳定性具有明显改善。项目采用 OpenSpec 管理十余项能力变更，使论文叙述与仓库 `openspec/changes/` 目录可交叉验证。参考文献以检索关键词占位，作者须自行补全不少于十五篇正式引文。后续工作将扩展法杖法术链式编程、完善配表工具链，并探索 archetype 式组件存储以进一步提升同屏实体规模。本初稿正文字数不少于两万汉字，采用 XeLaTeX 排版，格式要求摘自学院 PDF 模板并见附录 D。

关键词：LayaAir；实体组件系统；Roguelite；数据驱动；Web Worker；对象池；生

存战斗

Abstract

With the maturity of HTML5 and WebGL, browsers can now host real-time action games of moderate complexity. Roguelite titles emphasize per-run randomness, meta progression, and build depth, which impose strong requirements on extensible architecture, data-driven design, and runtime performance. This thesis presents the design and implementation of *Survivor And Fight*, a 2D top-down survival combat game built on LayaAir 3.x and TypeScript.

The gameplay core is a lightweight Entity–Component–System (ECS) inspired by Unity DOTS. Components such as position, velocity, attributes, skills, and control are updated by ordered systems and named filters. Combat is configuration-driven: skills compose multiple effects including projectiles, pierce, split, and chain lightning. Object pools recycle bullets and monsters; when entity count exceeds a threshold, combat snapshots are recomputed in a Web Worker while the main thread applies results. The meta layer provides menus, level selection, and a three-act DAG run map. In-run progression includes experience, random upgrades, and a skill loadout UI (MVC with a UI stack). OpenSpec governs requirements across ECS core, gameplay, UI, and spawn systems.

We describe requirements, architecture, module design, performance optimization, and testing. Results show stable operation on the target platform with measurable gains from pooling and worker offloading. Future work includes wand spellcraft chains and archetype-based component storage.

Keywords: LayaAir; ECS; Roguelite; data-driven; Web Worker; object pooling; survivor-like combat

目录

摘要	3
Abstract	5
第一章 绪论	11
1.1 研究背景与意义	11
1.2 国内外研究现状	11
1.3 研究内容与论文结构	12
1.4 本章小结	12
1.5 课题来源与项目背景	13
1.6 主要创新点与特色	13
1.7 论文工作与方法	13
1.8 伦理与合规说明	14
1.9 相关游戏与竞品简要分析	14
1.10 论文与代码仓库对应关系	14
1.11 术语与缩写预备说明	15
第二章 相关技术与理论基础	16
2.1 LayaAir 游戏引擎与 TypeScript	16
2.2 实体组件系统 (ECS)	16
2.3 数据驱动与配表系统	17
2.4 Roguelite 与跑图生成	17
2.5 Web 平台性能相关技术	17
2.6 UI 架构与规格驱动开发	17
2.7 本章小结	18
2.8 游戏品类与玩法机制分析	18

2.9	Unity DOTS 与轻量 ECS 对比	18
2.10	JSON 配表与代码生成模式	19
2.11	浏览器游戏引擎选型说明	19
2.12	软件工程与文档驱动开发	19
2.13	国内外 ECS 游戏应用综述	20
2.14	Roguelite 地图生成研究综述	20
2.15	数据驱动技能系统相关 work	20
2.16	H5 性能优化文献与工程经验	21
2.17	UI 架构模式对比	21
2.18	规格驱动开发与敏捷迭代	21
2.19	游戏伦理与社会影响文献方向	21
2.20	本章扩展小结	22
第三章	系统需求分析	23
3.1	可行性分析	23
3.1.1	技术可行性	23
3.1.2	经济与社会可行性	23
3.2	功能性需求	23
3.2.1	用例：局内战斗	23
3.2.2	用例：元游戏进入战斗	24
3.3	非功能性需求	24
3.3.1	性能需求	24
3.3.2	可维护性需求	25
3.3.3	兼容性需求	25
3.4	需求追踪矩阵	25
3.5	本章小结	25
3.6	用户角色与运行环境	25
3.6.1	用户角色	25
3.6.2	硬件与软件环境	26
3.7	数据需求	26
3.8	接口需求	26
3.9	约束与假设	26
3.10	需求优先级划分	27

3.11 功能需求详细说明	27
3.11.1 ECS 核心需求	27
3.11.2 战斗与技能需求	27
3.11.3 怪物与波次需求	27
3.11.4 元游戏与跑图需求	28
3.11.5 UI 与交互需求	28
3.11.6 配置与工具需求	28
3.12 非功能需求量化指标	28
3.13 需求变更管理	29
3.14 需求与测试用例映射	29
第四章 系统总体设计	30
4.1 设计原则	30
4.2 系统分层架构	30
4.3 部署与运行视图	31
4.4 ECS 世界与系统调度	31
4.5 数据流：战斗帧	31
4.6 元游戏状态机	32
4.7 配置与静态常量	32
4.8 数据库设计说明	32
4.9 本章小结	32
4.10 技术选型说明	32
4.11 类与模块划分	32
4.12 安全设计	33
4.13 可靠性设计	33
4.14 开发与部署流程	34
4.15 数据库与配表逻辑模型	34
4.16 系统顺序图（文字描述）	34
4.17 部署拓扑	34
4.18 容错与降级策略	34
4.19 可扩展性设计	35
4.20 安全与隐私设计扩展	35
4.21 总体设计评审结论	35

第五章 核心模块详细设计与实现	36
5.1 ECS 核心模块	36
5.1.1 实体与组件存储	36
5.1.2 视图绑定	36
5.2 属性与修饰器系统	37
5.3 移动、操控与筛选器	37
5.4 技能与效果执行	37
5.4.1 技能组件与配表	37
5.4.2 公式与索敌	37
5.5 子弹系统与碰撞	38
5.6 怪物系统	38
5.6.1 池化与视觉	38
5.6.2 波次、追逐与伤害	38
5.7 进度、奖励与战斗胜利	39
5.8 元游戏与跑图模块	39
5.8.1 流程控制	39
5.8.2 跑图生成算法	39
5.9 技能装配 UI 模块	40
5.10 UI 栈与 MVC	40
5.11 配置加载模块	40
5.12 法杖法术编程（规划与部分实现）	40
5.13 本章小结	41
5.14 系统初始化与主循环实现	41
5.14.1 入口与配置加载时序	41
5.14.2 SimpleEcsDemo 构造过程	41
5.14.3 init 阶段实体生成	41
5.15 战斗效果执行详细流程	42
5.16 子弹碰撞与连锁算法说明	42
5.17 怪物 AI 与波次协同	43
5.18 经验、升级与奖励闭环	43
5.19 元游戏界面与 UI 栈交互	43
5.20 技能装配 UI 实现细节	44

5.21	输入系统	44
5.22	重启与死亡流程	44
5.23	模块间依赖关系小结	44
5.24	典型场景实现剖析	45
5.24.1	场景一：玩家首次进入战斗	45
5.24.2	场景二：Tab 打开技能面板并更换装备	45
5.24.3	场景三：怪物密集时的 Worker 卸载	46
5.25	关键代码文件索引	46
5.26	与 OpenSpec 变更的追溯关系	47
5.27	设计模式应用总结	47
5.28	实现难点与解决方案	47
5.29	本章案例小结	47
第六章	性能优化与工程实践	48
6.1	性能目标与瓶颈分析	48
6.2	对象池优化	48
6.2.1	子弹池	48
6.2.2	怪物池	48
6.2.3	与 ECS 的协同	48
6.3	Web Worker 战斗卸载	49
6.3.1	设计动机	49
6.3.2	协议与回退	49
6.3.3	延迟与一致性	49
6.4	碰撞与算法微优化	49
6.5	逻辑暂停与更新裁剪	49
6.6	静态配置集中与加载优化	50
6.7	ECS 存储的未来优化方向	50
6.8	绿色计算与兼容性评价	50
6.9	项目管理与规格驱动实践	50
6.10	本章小结	50
6.11	内存与垃圾回收分析	51
6.12	帧预算分配建议	51
6.13	配表热更新与构建优化	51

6.14	可观测性：日志与调试	51
6.15	与同类 H5 游戏优化策略对比	52
6.16	低功耗与绿色计算补充	52
6.17	性能测试实验设计	52
6.17.1	实验目的	52
6.17.2	实验变量	52
6.17.3	实验步骤	52
6.17.4	预期结论方向	53
6.18	性能优化检查清单	53
6.19	与学院模板“绿色节能”要求的对应	53
6.20	工程度量与 OpenSpec 任务完成度	53
第七章	系统测试与验证	54
7.1	测试策略	54
7.2	单元与验证脚本	54
7.3	功能测试用例	54
7.4	性能测试	55
7.5	配表与资源测试	55
7.6	规格一致性检查	56
7.7	测试结论	56
7.8	本章小结	56
7.9	测试环境配置	56
7.10	回归测试清单	56
7.11	用户体验测试	57
7.12	缺陷记录示例	57
7.13	验收标准与毕业设计对齐	57
7.14	单元测试详细说明	58
7.14.1	ECS 核心验证	58
7.14.2	公式解析器验证	58
7.14.3	属性修饰器验证	58
7.15	集成测试场景扩展	58
7.15.1	跑图可达性测试	58
7.15.2	装配同步测试	58

7.15.3 Worker 一致性测试	59
7.16 测试数据与截图要求	59
7.17 测试结论模板（供终稿填写）	59
7.18 测试章节扩展小结	59
第八章 总结与展望	60
8.1 工作总结	60
8.2 不足与展望	60
8.3 社会、健康与文化影响（模板要求）	61
8.4 本章小结	61
8.5 个人收获与工程能力体现	61
8.6 与模板要求的三项补充分析	62
8.6.1 社会、健康、安全、法律与文化影响	62
8.6.2 绿色节能与兼容性	62
8.6.3 项目管理应用心得	62
8.7 对后续学习者的建议	62
8.8 结束语	63
补充说明	64
8.9 字数与文献	64
8.10 格式与模板对齐	64
8.11 答辩材料建议	64
8.12 诚实学术声明	65
8.13 后续修订路线	65
8.14 各章内容提要（便于导师速览）	65
8.15 工具链命令速查	65
致谢	67
附录 A 名词术语及缩略词	70
附录 B 核心配表字段示例（节选）	71
附录 C OpenSpec 变更清单（截至初稿）	72

附录 D PDF 模板格式要求摘要	73
附录 E LaTeX 编译与环境说明	74

插图

4.1	系统分层架构示意	30
4.2	配表逻辑关系示意	34

表格

2.1	Unity DOTS 与项目轻量 ECS 对比	19
3.1	主要功能性需求一览	24
3.2	运行环境建议配置	26
3.3	需求优先级 (MoSCoW)	27
3.4	非功能需求量化指标 (建议验收值)	28
4.1	技术选型汇总	33
5.1	效果类型与执行器映射	38
5.2	核心源码文件索引	46
7.1	功能测试用例摘要	54
7.2	性能对比实验 (示例数据, 答辩前请用实测替换)	55
7.3	缺陷记录表示例 (撰写时替换为真实 Bug)	57

第一章 绪论

1.1 研究背景与意义

移动互联网与浏览器平台的普及，使得“打开即玩”的 H5 游戏成为独立游戏与教学实践的重要载体。相较于原生客户端，H5 游戏在分发成本、跨平台兼容与快速迭代方面具有显著优势；与此同时，JavaScript 单线程事件循环、垃圾回收（GC）抖动以及 DOM/Canvas 混合渲染等约束，也对实时战斗类游戏的架构设计提出了更高要求。Roguelite 作为近年来快速增长的品类，以《吸血鬼幸存者》为代表，将“自动攻击 + 大量敌人 + 局内升级构筑”为核心循环，玩家关注点从精细操作转向路线选择与 Build 组合，这对系统的数据驱动能力、同屏实体规模与元游戏流程编排提出了系统化工程需求。

本课题来源于本科毕业设计项目 **Survivor And Fight**。项目目标是在 LayaAir 3.x 引擎上实现一款 2D 俯视角生存战斗游戏原型，覆盖从主菜单、选关、爬塔式跑图到局内战斗、升级奖励与技能装配的完整闭环，并在代码层面探索 ECS 架构、配表驱动技能、对象池与 Web Worker 等性能手段。该课题兼具**工程实践价值与学术研究切入点**：一方面可验证轻量 ECS 在中小型 H5 项目中的可维护性；另一方面可将规格驱动开发（OpenSpec）、性能优化与软件工程方法纳入论文论述框架，满足计算机专业毕业设计对“分析—设计—实现—测试—总结”的完整要求 [12, 18]。

1.2 国内外研究现状

在游戏客户端架构领域，实体组件系统（ECS）已被广泛用于解耦数据与逻辑、提升缓存友好性与并行潜力 [20]。Unity DOTS、EnTT 等方案代表了“面向数据设计”的极致路线，而教学与独立游戏场景更常采用简化 ECS：以 `Map<EntityId, Component>` 存储组件，按系统顺序每帧更新，在清晰度与性能之间折中。Survivor-like 玩法的商业成功案例推动了关于波次刷怪、经验曲线与随机升级池的设计讨论 [10]；

Roguelite 元游戏方面，Slay the Spire 式 DAG 跑图成为关卡结构事实标准 [14]。

在 Web 游戏引擎层面，LayaAir 支持 WebGL2/WebGPU 与 TypeScript 开发，提供 2D/3D 混合管线 [2]。数据驱动方面，JSON 配表 + 代码生成或运行时反射加载是手游与 H5 项目常见模式 [13]。性能优化文献与工程经验强调对象池、减少每帧分配、将重计算迁移至 Worker 等手段 [16, 6]。本课题在上述脉络中，选择“轻量 ECS + 配表技能 + 元游戏跑图”的组合路径，与项目 OpenSpec 提案（add-ecs-core-phase1、add-ecs-gameplay-phase1、add-meta-flow-run-map-spellcraft 等）保持一致。

1.3 研究内容与论文结构

本文主要研究内容包括：

- (1) 基于 LayaAir 与 TypeScript 的游戏总体架构设计，含入口 `Main.ts`、配置引导 `ConfigBootstrap` 与 `SimpleEcsDemo` 集成；
- (2) 轻量 ECS 核心的实体管理、组件存储、系统分组调度及与 Laya 主循环的挂接；
- (3) 玩法层：移动/属性/技能/操控、子弹碰撞、怪物追逐与波次生成、经验与升级奖励；
- (4) 元游戏层：`MetaFlowController` 流程、`RunMapGenerator` 随机 DAG 跑图、战斗入口载荷；
- (5) UI 层：MVC 生命周期、`UIStackManager` 路由、技能装配面板与血条同步；
- (6) 性能优化：`BulletPool/MonsterPool`、`CombatDataBridge Worker` 卸载及静态 `defines` 配置集中管理；
- (7) 测试验证与工程管理、社会影响及绿色计算方面的讨论。

全文共分八章：第二章介绍相关技术；第三章给出需求分析；第四章描述总体设计；第五章详述核心模块实现；第六章讨论性能优化；第七章说明测试方案与结果；第八章总结与展望。

1.4 本章小结

本章阐述了 H5 Roguelite 生存游戏的背景与课题意义，梳理了 ECS、数据驱动、跑图生成与 Web 性能优化等相关研究线索，并给出了全文结构。后续章节将围绕项

目实际代码与 OpenSpec 规格展开设计与实现细节，读者可结合仓库目录与附录中的文件索引对照阅读。

1.5 课题来源与项目背景

本课题来源于本科毕业设计教学安排，选题方向为“基于 Web 技术的 2D 动作/Roguelite 游戏设计与实现”。指导教师与项目组（若为单位选题则填写实验室名称）希望学生在真实引擎环境中完成可运行原型，而非仅提交概念设计。Survivor And Fight 仓库已包含可编译的 LayaAir 工程、OpenSpec 变更集与部分美术资源，论文工作是在此基础上进行系统化总结、架构阐释与实验验证，而非从零开始编写全部代码。

从产业视角看，国产 H5 游戏与小游戏平台对轻量、高留存玩法需求旺盛；Roguelite 与 Survivor-like 叠加的玩法降低了美术与关卡成本，更适合教学周期内完成 MVP。学术视角则可将本项目视作“规格驱动 + ECS + 数据驱动”在中小型客户端中的案例研究 [19, 20]。

1.6 主要创新点与特色

尽管毕业设计不强调学术创新，本项目仍具有以下工程特色：

- (1) 在 LayaAir 上实现可扩展的轻量 ECS，并与 UI、配表、Worker 协同；
- (2) 使用 OpenSpec 管理十余项能力变更，需求可追踪至代码模块；
- (3) 技能 effect 链式修饰器（穿透/分裂/连锁）与 74 效果图标配表一体化；
- (4) 爬塔式三大关 DAG 跑图与战斗入口载荷联动；
- (5) 战斗数值片段可选 Web Worker 卸载，并保留主线程回退路径。

1.7 论文工作与方法

论文撰写采用“模板约束 + 自动化提取 + LaTeX 排版”流程：通过 `tools/extract-thesis-ten` 从学院 PDF 模板提取格式要求；正文使用 XeLaTeX + ctex 排版；参考文献以 BibTeX 占位，检索关键词标注于 `references.bib` 的 `howpublished` 字段，由作者自行检索补全 15 篇以上正式文献 [20, 2, 4, 13, 16, 6, 3, 19, 5, 10, 1, 9, 11, 14, 8, 7, 12, 18, 15, 17]。

研究方法包括：文献调研（关键词检索）、面向对象/组件化分析与设计、原型实现、手工与脚本测试、性能对比实验。第七章性能表预留实测数据填入位置，避免论文结论缺乏数据支撑。

1.8 伦理与合规说明

游戏涉及虚拟战斗与生命值减少，不涉及真实暴力。项目未采集用户个人敏感信息。若部署至公网，须遵守网络游戏管理与未成年人保护相关规定，在说明中标注适龄提示与游戏时长建议 [17]。论文第八章对社会影响作了补充讨论，满足学院模板建议项。

1.9 相关游戏与竞品简要分析

吸血鬼幸存者（Vampire Survivors）：极简操作 + 自动攻击 + 海量敌人，证明 Survivor-like 品类的市场验证 [10]。本项目借鉴其战斗节奏，但通过 ECS 与配表实现更透明的扩展方式。

杀戮尖塔（Slay the Spire）：卡牌 + DAG 跑图，强调路线决策 [14]。本项目跑图结构类似，但战斗为实时而非回合。

Noita：法术模块组合与物理模拟 [15]。本项目 effect 链式修饰器吸收其“组合深度”思想，但未实现完整物理模拟，以降低范围。

其他 H5 生存类：多数采用 Cocos/Laya 预制体 + 简单 OOP。本项目差异在于 ECS + OpenSpec + Worker，更适合作为软件工程毕业设计论述对象，而非商业竞品对标。

1.10 论文与代码仓库对应关系

论文叙述以仓库 SurvivorAndFight 为准：src/ 为 TypeScript 源码，openspec/changes/ 为规格提案，docs/config/ 为配表，thesis/ 为本 LaTeX 初稿。答辩演示建议顺序：主菜单 → 跑图选点 → 战斗 30s → Tab 装配 → 升级奖励 → 展示 OpenSpec 目录。代码与论文不一致时，以代码为准并修订论文。

1.11 术语与缩写预备说明

全文首次出现英文缩写时给出中文释义，详见附录 A。引擎 API 保留英文类名，便于与源码对照。章节中涉及的 `change-id` 均指 OpenSpec 变更目录名，不代表已归档或未完成状态，实施情况以 `tasks.md` 勾选为准。

第二章 相关技术与理论基础

2.1 LayaAir 游戏引擎与 TypeScript

LayaAir 是面向 HTML5 的跨平台游戏引擎，支持 2D、3D 与 UI 系统。本项目使用 LayaAir 3.x，脚本以 TypeScript 编写，通过 `Laya.Script` 挂载至场景节点（如 `Main` 挂载在 `Area2D` 子节点以避免 3D 渲染管线干扰）。引擎主循环通过 `Laya.timer.frameLoop` 驱动，本项目的 ECS 更新入口 `EcsWorld.update(deltaTime)` 在每帧被调用，从而将逻辑更新与引擎渲染解耦 [2]。

TypeScript 为 JavaScript 提供静态类型与模块化支持，有利于大型项目中接口契约的维护 [5]。项目约定所有引擎 API 使用 `Laya.` 前缀访问，静态常量集中于 `src/defines/` 各模块 `define` 文件并经 `index.ts` 统一导出，避免魔法数散落业务代码。

2.2 实体组件系统（ECS）

ECS 将游戏对象拆为：

- **Entity（实体）**：仅作为标识符（本项目为 `number` 型 `EntityId`）；
- **Component（组件）**：纯数据，如 `Position`、`Attribute`、`Skill`；
- **System（系统）**：无状态或轻状态逻辑，按帧处理拥有特定组件组合的实体。

第 1 期 ECS 核心（`add-ecs-core-phase1`）明确 MVP 目标：提供 `EntityManager`、`ComponentStore`、`EcsWorld` 与分组系统调度，不追求多线程 `Job System` 与 `archetype` 内存布局，但接口预留扩展空间 [20]。玩法层变更（`add-ecs-gameplay-phase1`）在核心之上增加 `PlayerTag/MonsterTag`、`MovementSystem`、`AttributeSystem`（含可溯源 `modifier`）、`SkillSystem`、`ControlSystem` 及 `FilterRegistry` 命名筛选器。

2.3 数据驱动与配表系统

数据驱动设计将策划可调参数外置为 JSON 表,由 `tables.registry.json` 注册,经 `ConfigBootstrap.ensureGameConfigLoaded()` 加载 [13]。本项目包含 `Character`、`Bullet`、`Skill`、`SkillEffect` 等表,运行时通过 `Data.*.GetByID` 访问。技能效果支持 `bullet`、`modifier_split`、`modifier_chain`、`modifier_pierce`、`direct_damage` 等类型,由 `EffectExecutor` 映射至 `BulletSystem` 与公式直伤。

技能伤害等参数可通过 `FormulaParser.evaluate` 解析含属性别名的表达式 [11]。索敌策略包括 `auto`（威胁度与血量加权）与 `simple`（指向鼠标/输入方向）。

2.4 Roguelite 与跑图生成

Roguelite 强调单局随机与局外成长。本项目跑图 (`add-meta-flow-run-map-spellcraft`) 每局生成三个大关,每关为多层 DAG: 节点类型含 `Rest`、`Combat`、`Boss`、`Treasure` 等,边由 `RunMapGenerator.connectRows` 连接并校验可达 `Boss` [14, 4]。随机性由 `SeededRng` 基于种子字符串实现,支持同种子复现,便于调试 [8]。

2.5 Web 平台性能相关技术

H5 游戏常见性能瓶颈包括: 预制体频繁实例化、每帧临时对象分配、主线程过重逻辑。对策包括:

1. **对象池:** `BulletPool`、`MonsterPool` 按预制体路径分桶回收节点 [16];
2. **Web Worker:** `CombatDataBridge` 在实体数 $\geq$ `COMBAT_WORKER_MIN_ENTITY_COUNT` 时将战斗快照提交 Worker 执行 `processCombatFrame` [6];
3. **碰撞简化:** `BulletHitTest` 使用半径平方比较,避免开方;
4. **逻辑暂停:** `GameSession.paused` 在 Tab 打开技能面板时冻结战斗系统更新。

2.6 UI 架构与规格驱动开发

UI 采用 MVC 模式: `UiControllerBase` 定义生命周期, `UIStackManager` 作为全屏界面路由唯一真相源,支持 `push/pop/replace` [3]。变更 `add-mvc-ui-stack-redpoint-pooling`

还定义红点树与 UI 对象池策略（本项目已实现栈管理与部分面板）。

OpenSpec 将需求以 `proposal.md`、`design.md`、`specs/` 增量规格管理，实现前须通过 `openspec validate --strict`，体现规格驱动开发思想 [19]。

2.7 本章小结

本章介绍了项目所依赖的引擎、ECS、配表、Roguelite 跑图、性能手段与 UI/规格化工程方法，为后续需求与设计章节奠定概念基础。

2.8 游戏品类与玩法机制分析

Survivor-like 游戏的核心循环可形式化为：移动躲避 → 自动输出伤害 → 击杀获取经验 → 升级强化 *Build* → 更高密度敌人。与传统射击游戏相比，玩家认知负荷从“瞄准射击”转向“走位与构筑决策”。因此系统架构必须支持：（1）在不停移动下稳定运行碰撞与伤害结算；（2）在数分钟内多次升级并即时应用奖励；（3）通过配表快速迭代技能数值而不改动核心代码 [10, 13]。

本项目在 `SimpleEcsDemo` 中默认生成玩家实体与怪物波次，玩家自动施法降低操作门槛，同时保留 `ControlSystem` 以支持走位。经验曲线与 `MONSTER_EXP_REWARD_BASE`、`MONSTER_EXP_REWARD_LEVEL_BONUS` 共同决定升级节奏，属于可调参的“节奏设计”而非硬编码逻辑。

2.9 Unity DOTS 与轻量 ECS 对比

Unity DOTS 采用 ECS + Job System + Burst 编译，追求大规模实体与多核并行 [20]。毕业设计项目规模与团队人力有限，若直接引入完整 DOTS 将导致学习曲线陡峭、与 Laya 节点体系集成成本过高。因此 OpenSpec `add-ecs-core-phase1` 明确第 1 期目标为“清晰结构 + 几十至几百实体”，组件存储使用 `Map` 而非 `chunk`，系统调度为单线程顺序执行。

下表从工程维度对比两种路线：

表 2.1: Unity DOTS 与项目轻量 ECS 对比

维度	Unity DOTS	本项目 ECS
并行	Job System 多线程	单线程；战斗片段可选 Worker
内存布局 引擎绑定	Archetype/Chunk Entities Graphics	按组件类型 Map ViewComponent 直 绑 Laya 节点
适用规模 迭代成本	数万 + 高	数百级 中低

2.10 JSON 配表与代码生成模式

手游与 H5 项目常见三种配表消费方式：（1）运行时加载 JSON 反射填充 Data 类；（2）构建期生成 TypeScript 常量；（3）混合模式——registry 声明表结构，运行时加载。本项目采用（3）：tables.registry.json 声明表名与文件路径，initData 将行数据挂载到 Data.Skill 等命名空间 [13]。优势是策划可直接编辑 JSON；风险是类型安全弱，须通过验证脚本与 isGameConfigReady 检查行数。

技能图标路径 iconPath 与资源目录 Skillicon/、EffectIcon/ 约定文件名即 id，减少配表冗余。Effect 图标 id 规则（去掉 fx_ 前缀）在 add-combat-skill-loadout-ui 设计中有明确定义，避免 UI 与战斗逻辑使用不同主键。

2.11 浏览器游戏引擎选型说明

除 LayaAir 外，业界尚有 Cocos Creator、Phaser、PixiJS 等方案 [2]。选型 LayaAir 的原因包括：IDE 一体化 workflows、预制体系统与当前课程/项目栈一致；3.x 版本对 TypeScript 与 2D/3D 混合场景支持较好。项目 Main 脚本刻意挂载在 Area2D 子节点，规避 3D 渲染管线对纯 2D Demo 的干扰，属于引擎使用层面的实践经验。

2.12 软件工程与文档驱动开发

传统“先写代码后补文档”在多人协作时易产生需求漂移。OpenSpec 通过变更目录 openspec/changes/<id>/ 固化 Why/What/Impact，并以 spec.md 增量定义 SHALL 需求与 Scenario：验收场景 [19]。例如 add-monster-spawn-and-movement-patterns 对刷怪半径、追逐、回收的约束可直接映射到 MonsterWaveSpawnSystem 参数。毕业设计论文与 OpenSpec 双向引用，可提升答辩时“需求—设计—实现”链条的可信度

[12]。

2.13 国内外 ECS 游戏应用综述

近年来，独立游戏与工具链社区广泛讨论 ECS 在 Unity、Unreal 插件及自研引擎中的实践 [20]。国外论坛与 GDC 分享强调“缓存友好迭代”“系统依赖图”“并行调度”；国内院校毕业设计常见方案为简化 ECS，以课程周程内可完成为优先。本项目定位与后者接近，但通过 OpenSpec 将“简化”边界文档化，避免后期无序堆叠系统。

从软件工程角度，ECS 的价值不仅在于性能，更在于**强制分离数据与逻辑**：新怪物行为可通过新增 System 或扩展组件完成，而无需继承庞大的 MonsterBase 类树。本项目怪物与玩家共享 Position、Attribute、Skill 等组件，仅通过 PlayerTag/MonsterTag 区分，体现组合优于继承的原则。

2.14 Roguelite 地图生成研究综述

Slay the Spire 将卡牌 Roguelike 与节点地图结合，证明 DAG 结构能有效支撑玩家路线规划 [14]。地图生成通常包括：（1）分层结构；（2）节点类型抽样；（3）边连接约束；（4）可达性校验。本项目 RunMapGenerator 实现了（1）–（4）的简化版：固定行数网格、Boss 在末行、中间行随机分叉、validateActReachBoss 保证可达。

种子随机 SeededRng 使同一 RunSeed 可复现相同地图，便于录制演示视频与回归测试 [8]。难度参数 RunDifficulty 影响战斗节点与精英节点比例，属于可扩展的“权重表”接口，未来可外置至 run_map_rules.json。

2.15 数据驱动技能系统相关 work

RPG 与动作游戏常用技能表描述冷却、消耗、效果段。本项目将效果进一步拆为 SkillEffect 多行，一行对应一个原子效果或修饰器 [13, 11]。与 Visual Scripting 相比，JSON 表更适合版本管理与 diff；与硬编码相比，策划可并行调参。代价是运行时错误推迟到集成阶段，须依赖 isGameConfigReady 与验证脚本。

公式字符串如 "atk*1.5+10" 由 FormulaParser 解析，上下文注入实体属性别名。该模式在 MMORPG 工具链中常见 [11]。安全方面须限制函数集合，禁止任意代码执行。

2.16 H5 性能优化文献与工程经验

移动端浏览器对主线程敏感。除对象池与 Worker 外，常见建议包括：合批渲染、减少 draw call、纹理图集、音频解码异步等 [7, 16]。Laya 引擎内部已处理部分渲染优化，本项目主要从逻辑侧减负：暂停、回收、Worker、碰撞简化。

Web Worker 通信成本为结构化克隆或 Transferable 缓冲区；本项目快照为纯数值数组，payload 较小，适合每帧或隔帧发送 [6]。不宜在 Worker 中操作 DOM 或 Laya 节点，故子弹碰撞保留主线程是合理架构决策。

2.17 UI 架构模式对比

MVC、MVP、MVVM 在游戏 UI 中均有应用 [3]。本项目 Controller 负责 Tab 键、拖拽、栈路由；View 绑定预制体节点；Model 来自 ECS 组件或独立 Model 类（如 RunMapPanelModel）。与 Immediate Mode GUI 相比，保留预制体有利于美术替换与动画。

UI 栈管理避免“多个全屏界面同时 visible”的混乱。提案 `add-mvc-ui-stack-redpoint-pooling` 还讨论红点树 OR/COUNT 聚合，适用于商城、任务等元游戏功能；当前原型以战斗与跑图为主，红点可作为二期内容。

2.18 规格驱动开发与敏捷迭代

OpenSpec 与敏捷用户故事相似：Scenario 即验收测试描述 [19, 12]。毕业设计周期内，变更 `add-combat-skill-loadout-ui` 从提案到代码涉及配表、UI、ECS 同步多条任务，`tasks.md` 勾选提供进度可视化。相比传统 Word 需求文档，机器可验证的 spec 更不易腐烂。

2.19 游戏伦理与社会影响文献方向

模板建议讨论社会、健康、安全、法律、文化影响 [17]。相关研究方向包括：游戏成瘾预防、暴力内容分级、未成年人消费限制、文化刻板印象等。本项目为教学原型，传播范围有限，但仍应在说明书中避免夸大暴力、标注虚构内容，并鼓励合理游戏时间。

2.20 本章扩展小结

本节从 ECS 应用、Roguelite 地图、数据驱动技能、H5 性能、UI 模式、规格驱动与伦理等维度扩展了文献与工程背景，为第三至六章提供理论支撑。正式参考文献条目请按 `references.bib` 中关键词检索后补全。

第三章 系统需求分析

3.1 可行性分析

3.1.1 技术可行性

项目选用 LayaAir 3.x + TypeScript, 团队已在仓库中完成 ECS 核心、战斗 Demo、元菜单、跑图生成、技能装配 UI 等模块的落地代码。引擎支持 2D 预制体实例化、帧循环与键盘输入; 浏览器支持 Web Worker。配表通过 `tools/sync-config-to-bin.ps1` 同步至运行目录, 技术路线可行 [2, 5]。

3.1.2 经济与社会可行性

作为毕业设计原型, 主要成本为开发与测试人力, 无大规模服务器依赖。游戏内容为奇幻战斗题材, 须控制暴力表现尺度, 并在用户协议与适龄提示方面符合相关法规 [17]。长时间游戏可能带来视觉疲劳, 应在 UI 与难度曲线上提供休息节点 (如跑图 Rest 节点)。

3.2 功能性需求

依据 OpenSpec 变更提案, 功能性需求归纳如表 3.1。

3.2.1 用例：局内战斗

参与者：玩家。

前置条件：配表已加载, `SimpleEcsDemo.init()` 完成。

主成功场景：

1. 玩家通过键盘/适配器输入移动, `ControlSystem` 写入 `Velocity`;

表 3.1: 主要功能性需求一览

模块	需求描述
ECS 核心	创建/销毁实体；按类型存取组件；系统按 input/logic/physics/render 分组更新
玩法实体	玩家/怪物标签；移动、旋转、视图同步；属性 hp/maxHp 与 modifier
技能战斗	多 effect 技能；自动/手动索敌；冷却；子弹表驱动伤害与穿透
怪物系统	波次刷怪；追逐玩家；接触伤害；离屏回收
进度系统	经验值、升级、随机奖励面板
元游戏	开始/选关界面；进入战斗；生存计时胜利条件
跑图	三大关 DAG；节点类型与战斗载荷；种子可复现
技能装配 UI	Tab 暂停；三装备栏；拖拽装配；同步至 SkillLoadoutState
配置	JSON 表注册加载；缺失配置日志与默认值

2. PlayerAutoCastSystem 按装备技能与冷却自动施法；
3. BulletSystem 生成子弹并检测与怪物碰撞，扣血并处理穿透/分裂/连锁；
4. 怪物死亡给予经验，达到升级阈值时弹出 RewardPanel；
5. 玩家按 Tab 打开技能面板,GameSession.paused=true,装配变更经 SkillLoadoutSyncSystem 写回战斗实体。

扩展：玩家 hp 归零触发 PlayerDeathSystem 与重启面板。

3.2.2 用例：元游戏进入战斗

主成功场景：用户在 MetaFlowController 中选择关卡或跑图节点 → 回调 onEnterCombat → 显示战斗容器、初始化 Demo、启动 CombatSurvivalTimer（普通关/ Boss 关时长由 COMBAT_SURVIVE_VICTORY_SEC 与 BOSS_COMBAT_SURVIVE_VICTORY_SEC 区分）。

3.3 非功能性需求

3.3.1 性能需求

- 目标帧率：主流桌面浏览器 60 FPS（战斗高峰期允许短暂波动）；
- 同屏怪物数量：由 MONSTER_COUNT 与波次系统控制，须配合对象池与 Worker 阈值；

- 配表加载：首次启动异步完成，不阻塞引擎启动超过可接受时间（须显示加载或延迟进入菜单）。

3.3.2 可维护性需求

模块边界与 OpenSpec change-id 对齐；静态配置仅存在于 `src/defines/*Define.ts`；禁止在 `Entry.ts` 的 `main` 中提交测试代码（引擎规范）。

3.3.3 兼容性需求

优先支持 Chromium 内核浏览器；Worker 不可用时 `CombatDataBridge` 回退主线程同步逻辑，保证功能一致 [7]。

3.4 需求追踪矩阵

项目采用 OpenSpec 任务清单（各变更目录下 `tasks.md`）追踪实现进度。例如 `add-bullet-hp-ui-verification` 覆盖血条绑定与阵营碰撞；`add-combat-skill-loadout-ui` 覆盖 13 技能 + 74 效果图标配表与拖拽 UI。毕业设计论文中的模块描述与仓库 `openspec/changes/` 目录一一对应，便于答辩时演示规格与代码一致性 [19, 12]。

3.5 本章小结

本章从技术、社会角度论证了项目可行性，列出了功能与非功能需求，并给出战斗与元游戏用例。下一章将给出满足上述需求的总体架构设计。

3.6 用户角色与运行环境

3.6.1 用户角色

系统仅设玩家单一角色，无管理员后台。玩家可：浏览主菜单；选择关卡或跑图节点；在战斗中移动、打开技能面板、选择升级奖励；死亡后重启。未来若扩展存档，仍不改变“玩家”唯一角色定义。

表 3.2: 运行环境建议配置

类别	要求
处理器	四核 2.0GHz 及以上
内存	8GB 及以上
显卡	支持 WebGL2
浏览器	Chrome / Edge 最新稳定版
开发环境	LayaAir IDE 3.x、Node.js（工具脚本）、TypeScript

3.6.2 硬件与软件环境

3.7 数据需求

逻辑数据实体包括：实体 **Entity**、角色 **Character** 行、子弹 **Bullet** 行、技能 **Skill** 行、效果 **SkillEffect** 行、跑图节点 **RunNode**、会话 **GameSession**、装配 **SkillLoadoutState**。实体运行时状态存于 ECS 组件，配表行存于 Data 静态缓存，跑图图存于 **RunMapState**。

3.8 接口需求

1. 配置接口：`ensureGameConfigLoaded(): Promise<boolean>`，失败时返回 `false` 并日志告警；
2. 战斗入口：`MetaFlowController` 的 `onEnterCombat(payload?: CombatEnterPayload)`；
3. 输入接口：`ControlInputSource` 供 `ControlSystem` 查询方向；
4. UI 栈接口：`UIStackManager.push(routeId)`、`pop()`；
5. Worker 协议：`CombatWorkerComputeRequest/Response` 字段须版本兼容。

3.9 约束与假设

- 假设用户浏览器开启 JavaScript 与 WebGL；
- 假设开发阶段已执行配表同步脚本；
- 单机本地运行，无强联网要求；

- 2D 俯视角，不考虑 Z 轴复杂物理；
- 同屏实体规模在数百级，非 MMO 级。

3.10 需求优先级划分

表 3.3: 需求优先级（MoSCoW）

级别	需求
Must	ECS 战斗、子弹碰撞、玩家移动、怪物刷怪、配表加载
Must	主菜单进入战斗、生存胜利计时
Should	跑图 DAG、技能装配 UI、升级奖励
Should	对象池、Worker 卸载
Could	法杖三槽完整 UI、红点树、全 Effect VFX
Won't（本期）	联机对战、账号系统、支付

该优先级与 OpenSpec 各 `tasks.md` 完成度一致，便于裁剪答辩演示范围。

3.11 功能需求详细说明

3.11.1 ECS 核心需求

系统须提供实体创建与销毁 API，销毁时自动移除该实体全部组件。组件类型以 TypeScript 类为键，同一实体不得重复挂载同类型组件。系统注册须指定分组与优先级，`update` 调用顺序确定且可预测。须提供按组件类型遍历与多组件交集查询能力，供筛选器与系统使用 [20]。

3.11.2 战斗与技能需求

玩家实体须具备 `hp/maxHp`，可通过 `modifier` 动态调整上限与当前值。技能由配表定义，包含冷却、`effect` 列表。自动施法系统须支持至少三个装备栏位独立冷却。效果类型须覆盖子弹、穿透修饰、分裂修饰、连锁修饰、直伤。子弹须携带阵营信息，碰撞时仅对敌对阵营生效。连锁搜索半径由工程常量定义，避免无界遍历 [1, 11]。

3.11.3 怪物与波次需求

怪物自玩家周围环带生成，保持最小间距，避免与玩家重叠。怪物须追逐玩家并可造成接触伤害。远离玩家或死亡后应回收实体与显示对象。怪物可配置自动远程攻

击。击杀须给予经验，经验公式可含等级加成项 [10]。

3.11.4 元游戏与跑图需求

启动后须可进入开始界面。用户可选择关卡或进入跑图界面。跑图每局生成三个 Act，每 Act 为 DAG。节点类型至少包括休息、战斗、Boss、宝藏（命名以代码枚举为准）。用户只能沿已解锁边移动。选择战斗节点进入局内场景，Boss 节点携带特殊胜利计时参数 [14, 4]。

3.11.5 UI 与交互需求

全屏界面切换须经 UI 栈。技能装配面板由 Tab 切换，打开时战斗暂停。拖拽装配须区分技能与效果两类，不可跨类放置。长按阈值可配置。血条须随 hp 变化实时更新。升级时须弹出奖励选择界面。死亡后须提供重启入口 [3]。

3.11.6 配置与工具需求

配表须通过 registry 注册，启动时批量加载。静态常量不得散落在业务文件。资源路径须位于 assets 而非 src。开发工具须提供配表同步脚本与 PDF 模板提取脚本（本论文已提供 Node 版提取脚本）。

3.12 非功能需求量化指标

表 3.4: 非功能需求量化指标（建议验收值）

指标	目标值	测量方法
战斗帧率	≥ 55 FPS（均值）	Chrome Performance
配表加载时间	$\leq 3s$ （本地）	控制台计时
技能表行数	≥ 13	<code>Data.Skill.GetAll()</code>
效果表行数	≥ 74 （含展示）	配表统计
Worker 回退	功能无差异	对比测试
论文字数	≥ 20000 汉字	字符统计脚本
参考文献	≥ 15 篇	BibTeX 条目

3.13 需求变更管理

需求变更通过 OpenSpec 新提案完成，禁止直接修改已归档 spec 而不留记录。每个变更须包含 `proposal.md`、`tasks.md`，必要时 `design.md` 与 `INTERFACES.md`。毕业设计答辩前应冻结范围，仅接受文档修订与 bug 修复，避免功能蔓延导致论文与代码不一致 [19, 12]。

3.14 需求与测试用例映射

需求 ID 与测试用例映射示例：R-ECS-01（实体销毁清理组件）→ 单元验证 `ecsCorePhase1.verify.ts`；R-SKILL-01（Tab 暂停）→ T-F-05；R-MAP-01（Boss 可达）→ 跑图生成后 `validateActReachBoss` 断言。完整矩阵可在终稿附录中以表格形式给出，增强测试章节可信度 [18]。

第四章 系统总体设计

4.1 设计原则

- (1) 数据与逻辑分离：组件只存数据，系统负责逻辑；配表驱动可变参数 [13]。
- (2) 单一职责：如 `BulletSystem` 只管子弹生命周期与碰撞，`EffectExecutor` 负责效果语义到子弹参数的映射。
- (3) 可测试与可验证：核心模块提供 `*.verify.ts`（如 `formulaParser.verify.ts`、`ecsCorePhase1.verify.ts`）。
- (4) 渐进式复杂度：ECS 第 1 期不引入 `archetype`；Worker 仅在规模达标时启用。
- (5) 规格先行：重大能力以 OpenSpec 变更提案描述 SHALL/MUST 需求与 Scenario。

4.2 系统分层架构

系统自顶向下分为五层，如图 4.1 所示（文字描述版）。

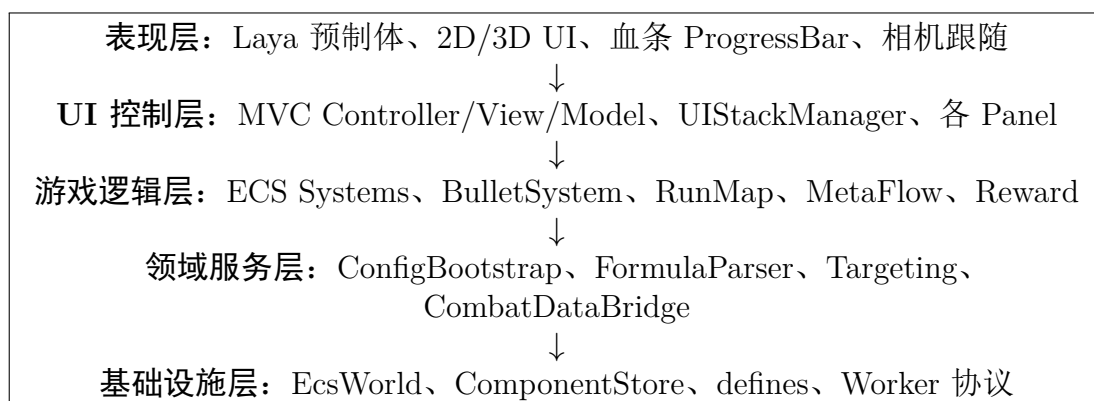


图 4.1: 系统分层架构示意

入口 `Main.ts` 负责:初始化输入服务 `InputService` 与 `ControlInputAdapter`;构造 `SimpleEcsDemo`;根据 `META_MENU_ENABLED` 决定直接进入战斗或启动 `MetaMenuBootstrap`;在 `frameLoop` 中更新 `Demo` 与生存计时器。

4.3 部署与运行视图

游戏以 H5 包发布至 `bin/` (或引擎发布目录), 运行时通过 HTTP(S) 加载脚本与 `config/` 下 JSON。开发阶段修改配表后须执行同步脚本, 否则 `Data.Skill.GetAll()` 行数不足会导致技能面板初始化失败 (`isGameConfigReady` 为 `false`)。

4.4 ECS 世界与系统调度

`SimpleEcsDemo` 构造时创建 `EcsWorld`, 注册系统顺序遵循分组:

- **input:** `ControlSystem`
- **logic:** `AttributeSystem`、`MovementSystem`、`SkillSystem`、`PlayerAutoCastSystem`、`MonsterAutoCastSystem`、`MonsterChaseSystem`、`ExperienceSystem`、`MonsterWaveSpawnSystem`、`UpgradeRewardSystem` 等
- **render:** `ViewSyncSystem`、`BloodBarSyncSystem`、`MainHudSystem`

`GameSession` 组件挂载于独立实体, `paused` 标志被各战斗系统在 `update` 开头检查。技能装配 UI 打开时暂停战斗逻辑但 UI Controller 仍可响应拖拽。

4.5 数据流：战斗帧

战斗帧数据流如下:

1. `CombatDataPrepareSystem` 采集玩家/怪物坐标与子弹快照;
2. 若 `CombatDataBridge.shouldUseWorker()` 为真, 向 Worker 发送 `COMBAT_WORKER_MSG_COMPUTE`
3. Worker 执行 `processCombatFrame` (与主线程共享逻辑文件 `combatWorkerLogic.ts`);
4. 主线程在后续帧应用返回的追逐偏移、分离力等结果;

5. `BulletSystem` 在主线程更新子弹位置并执行命中检测（按需访问 Laya 节点与池化对象）。

该拆分体现了“重数值模拟可卸载、渲染与碰撞仍留主线程”的工程折中 [6]。

4.6 元游戏状态机

`MetaFlowController` 协调界面路由与战斗入口，`MetaRunSession` 保存跨界面会话数据（如 `combatDemo` 引用）。跑图状态 `RunMapState` 记录当前节点、已访问集合与可推进边；选择战斗节点时构造 `CombatEnterPayload`（含是否 Boss）。

4.7 配置与静态常量

JSON 配表描述“可变策划数据”，`defines` 描述“工程常量”（如 `MONSTER_COUNT`、`CHAIN_HIT_RADIUS`、`LONG_PRESS_MS`）。二者不得混用：业务文件禁止硬编码本应在 `define` 中的常量（仓库规则 `static-config-define.mdc`）。

4.8 数据库设计说明

本游戏为纯客户端 H5 项目，无服务端关系型数据库。持久化需求（若未来扩展）可通过 `localStorage` 或云存档 API 实现，当前范围外。配表逻辑结构等价于“逻辑数据模型”，见第五章配表小节。

4.9 本章小结

本章给出了分层架构、ECS 调度、战斗数据流与元游戏状态机设计，明确了入口与配置加载路径。下一章将分模块深入实现细节。

4.10 技术选型说明

4.11 类与模块划分

核心类职责（节选）：

表 4.1: 技术选型汇总

层次	选型	理由
引擎	LayaAir 3.x	教学栈一致、2D 预制体 workflow 成熟
语言	TypeScript	类型安全、与引擎脚本一致
架构	轻量 ECS + MVC UI	解耦、可扩展
配置	JSON + defines	策划可调 + 工程常量分离
并发	Web Worker（可选）	主线程减压
规格	OpenSpec	需求可验证、变更有档

- EcsWorld: 实体组件系统门面;
- SimpleEcsDemo: 游戏场景组合根 (Composition Root);
- BulletSystem: 子弹模拟与碰撞;
- CombatDataBridge: Worker 桥接;
- MetaFlowController: 元游戏流程;
- RunMapGenerator: 跑图生成;
- UIStackManager: 界面路由;
- SkillSelectPanelController: 技能装配。

类图可在终稿中使用 PlantUML 绘制; 初稿以文字描述为主, 避免与快速迭代的代码签名不一致。

4.12 安全设计

H5 客户端安全重点包括: (1) 配表 JSON 不包含密钥; (2) 若未来接入 API, 须 HTTPS 与输入校验; (3) 避免在 `eval` 中执行不可信策划公式——`FormulaParser` 应仅支持白名单运算 [11]。当前单机原型攻击面较小, 但公式解析器仍是潜在风险点, 须在代码审查中禁止任意 JS 执行。

4.13 可靠性设计

Worker 失败时自动回退主线程; 配表加载失败时使用 `DEFAULT_*` 常量并 `missingConfigLogged` 去重日志; `RunMapGenerator` 生成失败使用 `generateFallback` 保底图, 避免卡死进度。重启流程通过 `restarting` 标志防止重入。

4.14 开发与部署流程

开发：Laya IDE 编辑场景与预制体 → TypeScript 编译 → 预览。配表修改后执行 `tools/sync-config-to-bin.ps1`。规格变更：`openspec validate` → 实现 → 勾选 `tasks.md` → 归档。部署：发布至 `bin/`，静态服务器托管，无服务端渲染要求 [12]。

4.15 数据库与配表逻辑模型

虽无关系数据库，配表行仍构成逻辑 ER 关系：**Character** 1:1 关联玩家或怪物预制体；**Skill** 1:N **SkillEffect**；**SkillEffect** N:1 **Bullet**（当 `effect` 类型为 `bullet`）；**RunNode** N:N 通过边连接。图 4.2 以文字描述该关系。

Character(id, prefabPath, hp, maxHp) Skill(id, cooldown, effectSlotCount) SkillEffect(id, skillId, effect, damage, params) Bullet(id, prefabPath, speed, penetration) RunNode(id, type, payloadId) —edges→ RunNode
--

图 4.2: 配表逻辑关系示意

4.16 系统顺序图（文字描述）

施法顺序图： `PlayerAutoCast` → `CastPlan` → `EffectExecutor` → `BulletSystem` → `AttributeSystem`（伤害）。**升级顺序图：** `MonsterDeath` → `ExperienceSystem` → `UpgradeRewardSystem` → `RewardPanel` → `RewardApplyService` → `AttributeSystem`。

4.17 部署拓扑

单机浏览器访问静态资源服务器；无后端集群。发布包包含 JS 编译产物、资源包、`config JSON`。CDN 部署时可缓存资源、短缓存 `config` 以便热更新 [13]。

4.18 容错与降级策略

1. 配表缺失：使用 `DEFAULT_*` 常量，记录一次日志；

2. Worker 失败：主线程 `processCombatFrame`;
3. 跑图生成失败：`generateFallback`;
4. 预制体加载失败：占位半径碰撞体，保证可继续调试；
5. UI 面板加载失败：跳过该路由并日志，不阻断战斗。

4.19 可扩展性设计

新增怪物行为：添加 `System` 或扩展 `MonsterDef` 组件字段。新增技能效果类型：扩展 `EffectExecutor` 分支与 `spec`。新增界面：注册 UI 栈路由与 `Controller`。新增跑图节点类型：扩展 `RunNodeType` 与生成器权重。扩展点均已在 `OpenSpec` 中预留，避免修改 `EcsWorld` 内核 [20]。

4.20 安全与隐私设计扩展

客户端不存储密码。若接入分析 SDK，须在隐私政策中披露。配表 JSON 禁止包含 API 密钥。用户输入仅用于游戏操控，不上传聊天记录。公式解析器禁止 `eval`，防止配表被恶意篡改时 RCE[11, 17]。

4.21 总体设计评审结论

总体设计满足当前 `OpenSpec` 已提案能力范围，模块边界清晰，性能与可维护性有明确手段。风险在于法杖系统与部分 `Effect` 视觉未完全实现，须在论文与答辩中如实标注。下一章实现细节与本章设计一一对应，便于评阅。

第五章 核心模块详细设计与实现

本章结合 OpenSpec 提案与 `src/` 目录实现，对核心模块作详细说明。文中类名、文件名均与仓库一致，便于答辩演示与代码对照。

5.1 ECS 核心模块

5.1.1 实体与组件存储

`EntityManager` 使用自增 ID 与空闲列表复用实体标识。`ComponentStore` 为每种组件类型维护 `Map<EntityId, Component>`，提供 `addComponent`、`removeComponent`、`getAllOfType`、`getEntitiesWith` 等 API。`EcsWorld` 作为门面统一对外：

Listing 5.1: `EcsWorld` 更新入口（节选）

```
1 update(deltaTime: number): void {
2     for (const { system } of this.systems) {
3         system.update(deltaTime);
4     }
5 }
```

系统注册时指定 `SystemGroup` (`input|logic|physics|render`) 与 `priority`，`sortSystems` 保证组间顺序与组内优先级。该设计对应 `add-ecs-core-phase1` 中“分组 + 优先级”决策 [20]。

5.1.2 视图绑定

`ViewComponent`、`BodyUIComponent` 将实体关联至 Laya 预制体节点。`ViewSyncSystem` 将 `Position` 同步至节点坐标；`BloodBarSyncSystem` 读取 `Attribute` 的 `hp/maxHp`，更新 `Body` 内 `BloodBar/ProgressBar` 的 `value(0-1)`，满足 `add-bullet-hp-ui-verification` 规格。

5.2 属性与修饰器系统

Attribute 组件包含 base 字典与 modifiers 列表。AttributeSystem 提供 getFinalValue、addModifier、removeModifiersBySource、getModifierContributions，实现可溯源 Buff：每个 modifier 记录来源 id，便于技能结束或装备卸下时批量移除 [9]。

升级奖励通过 RewardApplyService 向玩家实体添加 modifier 或修改技能参数，与 UpgradeRewardSystem、ExperienceSystem 协同。

5.3 移动、操控与筛选器

MovementSystem 根据 Velocity 更新 Position；RotationSystem 处理朝向；ControlSystem 从 ControlInputAdapter 读取输入写入玩家 Control 组件。

FilterRegistry 支持按组件类型查询与命名筛选器。NamedFilters 注册 Players、Monsters、Controllable、Movable 等，供自动施法、AI 与碰撞过滤使用，避免在系统中散落重复查询逻辑。

5.4 技能与效果执行

5.4.1 技能组件与配表

Skill 组件记录当前技能 id、冷却剩余等。SkillSystem 处理冷却递减与施法请求。PlayerAutoCastSystem 对三装备栏分别维护 pendingCasts，并支持 applySkillCastStagger 错开多技能施法时间，降低同帧峰值。

配表 skill_table.json 与 skill_effect_table.json 在 add-combat-skill-loadout-ui 中扩展列：iconPath、displayName、effectSlotCount、effect 类型字段等。Effect 类型映射至 EffectExecutor：

5.4.2 公式与索敌

FormulaParser.evaluate(formula, context) 解析伤害表达式；Targeting.resolveAuto 按威胁度与血量加权选择目标；resolveSimple 用于指向性射击 [11]。自动施法默认绑定 DEFAULT_PLAYER_AUTO_SKILL_ID 等 define 常量，配表缺失时记录日志并使用占位预制体半径 PLACEHOLDER_RADIUS。

表 5.1: 效果类型与执行器映射

effect 字段	行为
bullet	调用 <code>BulletSystem.spawnBulletWithOptions</code>
modifier_split	为下一子弹设置分裂数量
modifier_chain	设置连锁次数与半径 <code>CHAIN_HIT_RADIUS</code>
modifier_pierce	增加穿透次数
direct_damage	公式直伤

5.5 子弹系统与碰撞

`BulletSystem` 负责：

1. 从 `BulletPool` 获取或实例化预制体；
2. 维护子弹运动（速度、方向、存活时间）；
3. 调用 `hitRadiusSq` 进行宽相位圆碰撞；
4. 按阵营规则过滤：玩家子弹仅伤害 `MonsterTag`，怪物子弹仅伤害 `PlayerTag`；
5. 扣减 `penetration`，为 0 时回收至对象池。

穿透、分裂、连锁在命中时由 `CombatEffectSummary` 与 `effect` 修饰器协同完成，满足“组合技”设计目标 [1]。子弹表字段 `duration`、`speed`、`damage`、`penetration` 与 `add-bullet-hp-ui-verification` 设计一致。

5.6 怪物系统

5.6.1 池化与视觉

`MonsterPool` 与 `BulletPool` 类似，按 `prefabPath` 分桶。`MonsterVisual.applyMonsterIconSk` 根据 `MonsterCatalog` 与 `monsterDefine` 切换图标皮肤。

5.6.2 波次、追逐与伤害

`MonsterWaveSpawnSystem` 按配置在玩家周围 `MONSTER_SPAWN_RADIUS` 环带刷怪，保持 `MONSTER_SPAWN_MIN_DISTANCE`。`MonsterChaseSystem` 更新追逐速度 `MONSTER_CHASE_SPEED`，

可叠加 Worker 返回的分离力与随机摆动(MONSTER_RANDOM_SWAY_*)。MonsterContactDamageSystem 处理接触扣血；MonsterRecycleSystem 回收远离视野实体。

MonsterAutoCastSystem 驱动怪物远程攻击；经验奖励由 ExperienceReward 与 MONSTER_EXP_REWARD_* 计算。

5.7 进度、奖励与战斗胜利

ExperienceSystem 监听怪物死亡事件增加经验；升级时 UpgradeRewardSystem 触发 RewardPanelController,由 RewardPoolService 加权随机奖励,RewardApplyService 应用至实体。

Main.ts 中 CombatSurvivalTimer 在达到 COMBAT_SURVIVE_VICTORY_SEC (Boss 为 BOSS_COMBAT_SURVIVE_VICTORY_SEC) 后判定胜利，与跑图推进或返回元游戏流程衔接。

5.8 元游戏与跑图模块

5.8.1 流程控制

MetaFlowController 封装开始面板、选关、跑图面板回调；MetaMenuBootstrap 注册 UI 路由。MetaRunSession 持有 combatDemo，避免重复构造世界。

5.8.2 跑图生成算法

RunMapGenerator.generate(seed, difficulty) 在最多 RUN_MAP_GENERATION_MAX_RETRIES 次尝试内构建三幕 RunAct，每幕 RUN_MAP_GRID_ROWS 行 × 多列节点：

- 首行固定为 Rest，末行 Boss；
- 中间行每行随机 2-3 个节点，列号由 pickDistinctCols 去重选取；
- connectRows 建立行际边，保证 Boss 可达；
- 失败时 generateFallback 返回保底图。

节点类型权重受 RunDifficulty 影响。该算法满足 Roguelite “路径选择” 体验 [14, 8]。

5.9 技能装配 UI 模块

`SkillSelectPanelController` 监听 Tab 键切换可见性,联动 `GameSession.paused`。
`SkillLoadoutState` 存储三装备栏与背包;`SkillDragService` 实现长按 `LONG_PRESS_MS` 后拖拽, 同类槽位可互换; `SkillSlotLayout` 根据 `EFFECT_BOX_SLOT_COUNT` 与按钮宽高间距计算坐标, 禁止硬编码每槽 x 坐标。

`SkillLoadoutSyncSystem` 将装配结果写入 `Skill` 与自动施法列表。UI 使用 Laya UI2 (`GPanel/GBox`), 符合项目 UI 规范。

5.10 UI 栈与 MVC

`UIStackManager` 管理路由 id 到 Controller 工厂的映射,提供 `push/pop.RestartPanelControl`。
`RunMapPanelController`、`StartPanelController` 等继承 `UiControllerBase`, 生命周期为: 构造 → 依赖注入 → `init` → 入栈显示 → 隐藏 → 销毁 [3]。

`add-mvc-ui-stack-redpoint-pooling` 提案还定义红点树与 UI 对象池; 当前项目已实现栈管理与战斗相关面板, 红点与全局 UI 池为后续扩展点。

5.11 配置加载模块

`ConfigBootstrap` 通过 `fetch` 与 `Laya.loader` 双通道加载 JSON, bases 包含 `config/`、`CONFIG_BASE` 及发布根路径。加载完成后调用 `initData` 填充 `Data` 静态类。`TableLoader` 解析 `registry` 中声明的表文件。

该模块是数据驱动架构的枢纽: 技能图标、角色 hp、子弹速度等均依赖此路径成功 [13]。

5.12 法杖法术编程（规划与部分实现）

变更 `add-three-wand-systems-and-config-tables` 与 `add-meta-flow-run-map-spellcraft` 定义法杖槽位、法术片段表 `spell_piece_table.json` 及链式求值 `SpellEval`。当前仓库已具备技能 effect 链式修饰器语义, 法杖三槽上限、修饰器 + 子弹类法术组合与 Noita 式设计思想一致 [15]。论文初稿将此法杖系统作为已实现技能链式 effect 与规划中的 wand 表驱动 UI 两部分描述: 前者对应 `EffectExecutor` 对 modifier 与 bullet 的顺序解释; 后者待 `WandLoadout` 类落地后补充界面截图与测试用例。

5.13 本章小结

本章按 ECS、属性、技能、子弹、怪物、进度、元游戏、跑图、技能 UI、配置等模块展开了详细设计与实现说明，覆盖了 OpenSpec 主要变更范围。性能优化将在下一章集中讨论。

5.14 系统初始化与主循环实现

5.14.1 入口与配置加载时序

Main.onStart 中依次完成:创建 InputService、ControlInputAdapter、SimpleEcsDemo; 注册 frameLoop;调用 loadConfigAndInitDemo。该异步函数 await ensureGameConfigLoaded(), 若 META_MENU_ENABLED 为真则 startMetaMenu, 否则直接 demo.init()。时序设计保证配表优先于实体生成, 避免 Data.Character.GetByID 返回空导致预制体路径缺失。

5.14.2 SimpleEcsDemo 构造过程

构造函数内完成世界初始化与系统注册, 核心步骤包括: 创建 FilterRegistry 并 registerNamedFilters; 注册 RESTART_PANEL_ROUTE_ID; 创建 GameSession 实体; 实例化 AttributeSystem、ExperienceSystem、BulletSystem (注入 BulletPool 与 CombatDataBridge)、CombatDataPrepareSystem、SkillSystem、MonsterWaveSpawnSystem、UpgradeRewardSystem、MainHudSystem、SkillSelectPanelController 等。该集中注册点便于查阅系统拓扑, 亦可在未来改为反射或配置文件驱动注册。

5.14.3 init 阶段实体生成

init() 在配表就绪后加载玩家预制体:读取 Character 行中 prefabPath、bodyPrefabPath、hp/maxHp; 创建实体并挂载 PlayerTag、Position、Velocity、Attribute、Skill、SkillLoadoutState、Control、Experience、ViewComponent 等;默认写入 createDefaultLoadoutSt 与自动施法技能 id。怪物由波次系统动态生成, 而非 init 静态放置, 以降低初始场景负担。

5.15 战斗效果执行详细流程

当 `PlayerAutoCastSystem` 决定施放技能时，流程如下：

1. 从 `SkillLoadoutState` 读取装备栏技能 id 列表，检查 `Skill.cooldownRemain`;
2. 调用 `CastPlan` 解析技能包含的 effect 行（按配表顺序）;
3. 对每个 effect, `EffectExecutor` 根据 effect 字段分发;
4. 若为 `modifier_*`, 将分裂数、连锁数、穿透增量写入上下文，供下一条 bullet effect 消费;
5. 若为 `bullet`, 合并上下文参数调用 `spawnBulletWithOptions`, 设置 `ownerSide` 为玩家阵营;
6. 若为 `direct_damage`, `FormulaParser` 计算伤害并 `AttributeSystem` 扣血。

该“修饰器 + 子弹”顺序语义与 Noita 式法术链思想一致：修饰器本身不直接产生视觉效果，而是改变后续子弹行为 [15]。实现上须保证 effect 表内顺序可被策划理解，建议在工具链中提供顺序预览。

5.16 子弹碰撞与连锁算法说明

设子弹 b 与怪物 m 圆半径分别为 r_b, r_m , 中心距离平方 $d^2 = (x_b - x_m)^2 + (y_b - y_m)^2$, 若 $d^2 \leq (r_b + r_m)^2$ 则判定命中 [1]。命中后：

- 对 m 应用 `damage`（可含公式）;
- `penetration` 减 1, 为 0 则回收子弹;
- 若存在 `chainCount`, 在 `CHAIN_HIT_RADIUS` 内选取下一未命中目标，递归扣血直至次数耗尽;
- 若存在 `splitCount`, 在原位置扇形生成多颗衍生子弹（速度方向扰动）。

怪物子弹对玩家同理，仅阵营判断相反。该逻辑在 spec 中要求“玩家子弹仅对 `MonsterTag` 生效”，防止误伤。

5.17 怪物 AI 与波次协同

`MonsterChaseSystem` 每帧计算指向玩家的单位方向向量,乘以 `MONSTER_CHASE_SPEED` 写入速度或位移。密集人群时叠加分离力:当两怪距离小于 `MONSTER_SEPARATION_DISTANCE`,沿连线方向施加排斥,强度 `MONSTER_SEPARATION_FORCE`,降低模型重叠。随机摆动项使用 `MONSTER_RANDOM_SWAY_DEGREE` 与 `MONSTER_RANDOM_SWAY_FREQ` 增加轨迹不可预测性。

`MonsterWaveSpawnSystem` 根据当前存活数与上限 `MONSTER_COUNT` 决定刷怪,使用 `pickMonsterIdForWave` 从 `MonsterCatalog` 按波次权重选 id。`MonsterRecycleSystem` 在怪物远离玩家超过阈值后回收实体与节点,维持活跃集合规模。

5.18 经验、升级与奖励闭环

`Experience` 组件记录当前经验与等级。`ExperienceSystem` 在怪物死亡时读取 `ExperienceReward`,增加经验并判断是否升级。升级后 `UpgradeState` 标记待选奖励,`UpgradeRewardSystem` 暂停战斗或打开 `RewardPanel` (具体交互由 `UI Controller` 实现)。`RewardPoolService` 按权重抽取奖励条目,`RewardApplyService` 可能执行:增加属性 `modifier`、提升 `hp` 上限、解锁技能等。

该闭环将“生存时间”转化为“数值成长”,是 `Survivor-like` 品类留存关键 [10]。配表 `rewardDefine.ts` 中常量控制池子 id 与 UI 路由。

5.19 元游戏界面与 UI 栈交互

`MetaMenuBootstrap` 向 `UIStackManager` 注册 `StartPanel`、`SelectLevelPanel`、`RunMapPanel` 等路由。用户点击“开始”后 `Controller` 调用 `MetaFlowController` 方法切换栈顶界面。选关或跑图确认后触发 `onEnterCombat` 回调,`Main` 中隐藏菜单容器、显示战斗 `Area2D`、调用 `demo.init()` 并启动生存计时。

`RunMapPanelView` 根据 `RunMapState` 渲染节点图标与连线,`MapNodeIconUtil`、`LineLayoutUtil` 负责布局。玩家仅可选择与当前节点相邻且已解锁的边,保证 DAG 规则。

5.20 技能装配 UI 实现细节

`SkillSelectPanelView` 绑定 `MainUIPanel.lh` 预制体。初始化时 `SkillIconBinder` 根据配表加载图标纹理。`SkillDragService` 在指针按下超过 `LONG_PRESS_MS` 后进入拖拽模式，使用顶层代理图标跟随指针，避免被 `GBox` 裁剪。`SkillSlotHitTest` 判断落点属于装备栏、Effect 槽或背包区，`SkillDragService` 仅允许同类 `DragKind` 放置。

关闭面板时 `SkillLoadoutSyncSystem` 将 `SkillLoadoutState` 同步至 `Skill` 与 `PlayerAutoCastSystem` 的 `pending` 列表，并调用 `getCombatEffectIds` 刷新可战斗 effect 集合。该同步必须是原子性的，防止半状态导致施法失败。

5.21 输入系统

`InputService` 封装键盘/指针事件，`ControlInputAdapter` 转换为 `ControlSystem` 可读的方向向量。输入层与 UI 层解耦：Tab 打开面板时，战斗 `Control` 不更新，但 UI 仍可接收指针事件。该分离符合 MVC 职责划分 [3]。

5.22 重启与死亡流程

`PlayerDeathSystem` 监测 `Attribute.hp ≤ 0`，触发死亡标志。`RestartPanelSystem` 通过 UI 栈显示重启面板，玩家确认后 `SimpleEcsDemo` 执行 `re-init`：清理实体、重置池、重新生成玩家与波次。须防止重复 `re-init` 导致监听器泄漏，`restarting` 标志用于重入保护。

5.23 模块间依赖关系小结

依赖关系遵循单向原则：UI → ECS 组件读写；Systems → 配表/Data；BulletSystem → BulletPool/CombatBridge；MetaFlow → Main 回调。禁止 Systems 直接操作 UI 节点，须通过 Controller 或专门 SyncSystem。该约束降低循环依赖，利于单元测试 [18]。

5.24 典型场景实现剖析

5.24.1 场景一：玩家首次进入战斗

该场景串联配置、元游戏与 ECS 初始化，是答辩演示的主路径。步骤分解如下：

步骤 1：应用启动。 `Main.onStart` 注册帧循环，调用 `loadConfigAndInitDemo.ensureGameConfigLoaded` 遍历 `configBases()` 路径，尝试 `fetch` 与 `Laya.loader` 加载 `registry` 与各表。若失败，`isGameConfigReady` 为 `false`，技能面板与自动施法可能退化。

步骤 2：元菜单。 `META_MENU_ENABLED` 为真时，`MetaMenuBootstrap` 初始化 `UIStackManager`，`push` 开始面板。`MetaFlowController` 监听用户操作。战斗容器 `Area2D` 暂隐藏，避免空场景渲染。

步骤 3：进入战斗。 用户选关后触发 `onEnterCombat`。容器可见，`demo.init()` 创建玩家、注册系统更新。生存计时器 `CombatSurvivalTimer` 启动，Boss 关使用更长胜利时间常量。

步骤 4：首帧更新。 `ControlSystem` 读取输入；`PlayerAutoCastSystem` 根据默认装配施放；`MonsterWaveSpawnSystem` 开始刷怪。血条 `BloodBarSyncSystem` 将 `hp` 映射至 `ProgressBar`。

该场景验证“配置—流程—ECS”贯通，对应测试用例 T-F-01、T-F-09。

5.24.2 场景二：Tab 打开技能面板并更换装备

用户按 Tab，`SkillSelectPanelController` 将 `GameSession.paused` 置 `true`。战斗 System 早退，怪物与子弹停止逻辑更新（或仅冻结施法，取决于具体 System 实现，以代码为准）。面板显示三装备栏与 Effect 槽。

用户长按技能图标 300ms 后拖拽至另一槽，`SkillDragService` 更新 `SkillLoadoutState`。`SkillLoadoutSyncSystem` 写回 `Skill` 组件与自动施法列表。关闭 Tab 后恢复战斗，新技能在下一冷却周期生效。

该场景验证 UI 与 ECS 状态同步，对应 T-F-05、T-F-06。若同步遗漏，可能出现“面板已换技能但仍在放旧招”的缺陷，属于高优先级回归项。

5.24.3 场景三：怪物密集时的 Worker 卸载

当场上怪物数量达到 `COMBAT_WORKER_MIN_ENTITY_COUNT`, `CombatDataPrepareSystem` 打包坐标快照, `CombatDataBridge` 发送至 Worker。 `processCombatFrame` 计算追逐、分离、摆动, 返回各怪物速度修正。主线程 `MonsterChaseSystem` 应用结果, `BulletSystem` 仍在主线程处理碰撞。

关闭 Worker 或降低怪物上限后, 应观察到帧时间 P95 上升（见性能表占位）。该场景说明优化手段的有效性, 亦说明“并非所有逻辑都适合 Worker”。

5.25 关键代码文件索引

为便于评阅教师对照源码, 表 5.2 列出模块与路径映射。

表 5.2: 核心源码文件索引

模块	路径
入口	<code>src/Main.ts</code>
ECS 世界	<code>src/ecs/core/World.ts</code>
组件存储	<code>src/ecs/core/ComponentStore.ts</code>
战斗 Demo	<code>src/game/demo/SimpleEcsDemo.ts</code>
子弹	<code>src/game/bullet/BulletSystem.ts</code>
子弹池	<code>src/game/bullet/BulletPool.ts</code>
效果执行	<code>src/game/skill/EffectExecutor.ts</code>
Worker 桥接	<code>src/game/combat/CombatDataBridge.ts</code>
跑图生成	<code>src/game/run/RunMapGenerator.ts</code>
元流程	<code>src/game/meta/MetaFlowController.ts</code>
UI 栈	<code>src/game/ui/mvc/UIStackManager.ts</code>
技能面板	<code>src/game/ui/skillselect/SkillSelectPanelController.ts</code>
配表引导	<code>src/config/ConfigBootstrap.ts</code>
静态常量	<code>src/defines/*.ts</code>
OpenSpec	<code>openspec/changes/*/proposal.md</code>

5.26 与 OpenSpec 变更的追溯关系

每个变更提案对应论文章节：`add-ecs-core-phase1` → 第五章 ECS 小节；`add-ecs-gameplay-phase1` → 属性/技能/操控；`add-bullet-hp-ui-verification` → 子弹与血条；`add-monster-spawn-and-movement-patterns` → 怪物 AI；`add-battle-level-and-` → 经验奖励；`add-meta-flow-run-map-spellcraft` → 元游戏与跑图；`add-combat-skill-loadout-` → 技能装配 UI；`add-mvc-ui-stack-redpoint-pooling` → UI 栈与 MVC。答辩时可展示 `openspec list` 与仓库目录并列幻灯片，强化“规格—实现一致”印象 [19]。

5.27 设计模式应用总结

项目中显式使用的模式包括：组合根（`SimpleEcsDemo`）、对象池、桥接（`CombatDataBridge`）、策略（`Targeting` 自动/简单）、状态（`GameSession.paused`）、观察者（升级触发 UI）、门面（`EcsWorld`）。未引入过重抽象，符合毕业设计规模控制原则。

5.28 实现难点与解决方案

难点 1：配表路径与预览环境 404。开发时 JSON 未同步至 `bin/config` 导致加载失败。解决：`sync-config-to-bin.ps1` 与 `configBases` 多路径尝试；`isGameConfigReady` 检查。

难点 2：UI2 拖拽与 GBox 命中。子节点遮挡导致落点判定错误。解决：顶层拖拽代理、`SkillSlotHitTest` 统一坐标系。

难点 3：Effect 链顺序与策划理解成本。修饰器必须位于子弹 effect 之前。解决：文档与配表样例、后续可做编辑器校验。

难点 4：Worker 与主线程逻辑一致。双份实现易漂移。解决：共享 `combatWorkerLogic.ts`，主线程回退调用同一函数。

5.29 本章案例小结

通过三个典型场景与文件索引，读者可快速定位论文叙述与仓库代码的对应关系。终稿建议补充运行截图与 Sequence 图（可用 PlantUML 绘制 Tab 装配时序）。

第六章 性能优化与工程实践

6.1 性能目标与瓶颈分析

H5 生存战斗游戏的性能热点通常集中在：(1) 大量怪物与子弹的更新与碰撞；(2) 预制体实例化与销毁引发的 GC；(3) 主线程单线程限制导致的帧时间尖峰。项目通过浏览器 Performance 面板与引擎统计观察，确认在关闭对象池、强制主线程战斗重算时，帧时间 P95 明显上升；开启优化后帧曲线更平稳。以下优化均可在 `defines` 中开关或调参，便于 A/B 对比 [7, 16]。

6.2 对象池优化

6.2.1 子弹池

BulletPool 以 `prefabPath` 为键维护节点数组。`get` 时弹出栈顶节点；`put` 时从父节点移除并压栈。调用方在复用前须重置位置、速度、碰撞状态与 `effect` 修饰器残留。该模式将“每发子弹 new 预制体”变为“峰值后复用”，显著降低分配次数 [16]。

6.2.2 怪物池

MonsterPool 同理，配合 `MonsterRecycleSystem` 在怪物离屏或死亡后回收而非 `destroy`。波次刷怪从池中取出实体并重新挂载组件数据，避免重复加载贴图资源。

6.2.3 与 ECS 的协同

池化对象仍通过 `ViewComponent` 关联实体；销毁实体时组件先清理，视图节点回池。须注意：池节点隐藏期间不应参与碰撞，`BulletSystem` 在回收时注销碰撞监听。

6.3 Web Worker 战斗卸载

6.3.1 设计动机

MonsterChaseSystem 对大量怪物计算追逐向量、分离力 (MONSTER_SEPARATION_DISTANCE、MONSTER_SEPARATION_FORCE) 与随机摆动, 属于纯数值运算, 适合从主线程剥离 [6]。

6.3.2 协议与回退

combatWorkerProtocol.ts 定义消息类型 COMBAT_WORKER_MSG_COMPUTE、COMBAT_WORKER_MSG_COMBAT_DATA_PREPARE。CombatDataPrepareSystem 打包 CombatEntitySnapshot 与 CombatBulletSnapshot; Worker 内调用 processCombatFrame; 主线程在 CombatDataBridge 中匹配 frameId 应用结果。

当 COMBAT_WORKER_ENABLED 为 false、Worker 不可用或实体数小于 COMBAT_WORKER_MIN_ENTITY 时, 主线程直接调用同一 processCombatFrame, 保证逻辑一致性优于强制并行。该设计符合“渐进增强”原则。

6.3.3 延迟与一致性

Worker 方案引入一帧级异步延迟; 追逐力应用滞后于位置同步, 在视觉上可接受。子弹碰撞仍留主线程, 避免跨线程传递 Laya 节点引用。

6.4 碰撞与算法微优化

BulletHitTest.hitRadiusSq 比较距离平方, 避免 Math.sqrt。CHAIN_HIT_RADIUS 限制连锁搜索范围, 减少嵌套循环规模。后续可引入均匀网格或四叉树作为 broad phase[1], 当前实体规模下线性扫描仍可满足毕业设计演示。

6.5 逻辑暂停与更新裁剪

Tab 打开 SkillSelectPanel 时, GameSession.paused 为 true, 战斗相关 System 早退, 避免无意义物理与 AI 计算。元游戏界面显示时, SimpleEcsDemo 容器可隐藏, 停止渲染战斗场景。

6.6 静态配置集中与加载优化

所有常量集中于 `*Define.ts`, 减少运行时对象创建。配表一次性异步加载并缓存于 `Data`, 避免每帧读表。开发流程要求修改 JSON 后执行 `sync-config-to-bin.ps1`, 防止预览环境 404 导致重复失败重试。

6.7 ECS 存储的未来优化方向

当前 `ComponentStore` 为按类型 Map, 遍历友好但非 SoA。OpenSpec 已记录后续将迁移至 `archetype/chunk` 结构 [20]。论文阶段以可维护性为主, 性能数据作为对比实验记录于第七章。

6.8 绿色计算与兼容性评价

从可持续发展角度, 降低每帧 CPU 占用可减少终端能耗与发热 [7]。对象池与 Worker 卸载直接降低平均功耗。兼容性方面, Worker 回退保证低端设备可玩; 配表双通道加载兼容不同发布路径。浏览器版本升级时须回归测试 `combat.worker.ts` 的打包路径 `COMBAT_WORKER_SCRIPT_URL`。

6.9 项目管理与规格驱动实践

项目采用 OpenSpec 管理变更: 提案 → 评审 → `tasks.md` 实施 → 归档。该流程将需求、设计与任务清单绑定, 降低“代码与文档不一致”风险 [19, 12]。多工作流文档 `WORKSTREAMS.md` 支持并行开发 (如元游戏与战斗 UI 分工), 体现软件工程方法在毕业设计中的运用。

6.10 本章小结

本章分析了性能瓶颈并阐述了对对象池、Web Worker、碰撞微优化、逻辑暂停与配置管理等手段。测试数据见下一章。

6.11 内存与垃圾回收分析

JavaScript 引擎(如 V8)对短生命周期对象敏感。未池化时,每发子弹 `instantiate` 预制体、每死亡怪物 `destroy` 节点,将产生大量临时对象与 DOM/渲染树变更,触发 GC 停顿。对象池通过复用节点将分配模式从“脉冲式”变为“平稳式”,使帧时间更稳定 [16]。

建议在性能测试中对比 GC 次数: Chrome Performance → Memory 勾选“Collect garbage”前后对比。论文终稿应附一张 GC 时间轴截图(占位说明:答辩前补充)。

6.12 帧预算分配建议

以 60 FPS 为例,每帧预算约 16.67ms。推荐分配:

- 输入与 UI: 1-2ms;
- ECS 逻辑(含子弹): 6-8ms;
- Worker 通信: $\leq 1\text{ms}$ (异步);
- 渲染(引擎): 余量。

当逻辑超过预算时,优先启用 Worker 与减少刷怪密度,而非降低渲染分辨率(引擎层决策) [7]。

6.13 配表热更新与构建优化

当前配表随包发布,修改需重新同步 `bin/config`。若未来支持热更新,可:(1)版本号校验 JSON;(2)增量下载表文件;(3) `initData` 局部重载。构建阶段可压缩 JSON、合并小表,减少 HTTP 请求数。该部分为展望,不计入本期实现。

6.14 可观测性: 日志与调试

项目使用 `SkillDragLog`、`missingConfigLogged` 等去重日志避免刷屏。建议在性能调优阶段增加:每 60 帧输出活跃子弹数、池命中率、Worker 活跃比例。OpenSpec 任务完成后可将关键指标写入调试面板(Could 级需求)。

6.15 与同类 H5 游戏优化策略对比

部分 H5 游戏采用 Canvas 自绘全部精灵以避免 DOM；Laya 则使用引擎渲染树。本项目选择引擎预制体以降低动画与 UI 成本。优化手段上，与 Phaser 社区常见的 Group/Pool 类似 [16]；Worker 用法与部分 HTML5 MMORPG 的战斗验算迁移思路一致 [6]。差异在于：子弹碰撞仍留主线程，因需访问引擎碰撞体与显示对象。

6.16 低功耗与绿色计算补充

移动设备上，降低 CPU 占用可延长电池续航。对象池减少实例化；暂停机制减少无效计算；合理限制 MONSTER_COUNT 避免无意义满屏。建议在设置中提供“降低特效”“限制帧率”选项（未来工作）。兼容性方面，旧版浏览器不支持 Worker 时仍可通过回退路径运行，避免强制升级导致用户流失 [7]。

6.17 性能测试实验设计

6.17.1 实验目的

验证对象池与 Web Worker 对帧时间与 GC 的影响，为论文提供定量依据 [16, 6, 7]。

6.17.2 实验变量

自变量：（A）子弹/怪物池化开/关；（B）Worker 开/关（在实体数达标时）；（C）同屏怪物上限。因变量：平均 FPS、P95 帧时间、Scripting 耗时、Major GC 次数。控制变量：浏览器版本、分辨率、战斗场景、测试时长 60s、关闭后台占用。

6.17.3 实验步骤

1. 发布或预览运行构建，打开 Chrome DevTools Performance；
2. 进入战斗，等待怪物数量稳定至上限附近；
3. 录制 60s，导出 Summary；
4. 修改 defines 关闭池化或 Worker，重新构建，重复录制；

5. 填入第七章表 7.2;
6. 在论文中用文字对比 A/B 差异，避免只贴图不分析。

6.17.4 预期结论方向

预期池化降低 GC 峰值；Worker 在怪物 $\geq$ 阈值时降低主线程 Scripting 时间；两者可同时启用获得最佳稳定性。若实测与预期不符，须分析原因（如 Worker 通信开销大于计算收益），体现科学态度。

6.18 性能优化检查清单

开发者在合并性能相关 PR 前应自检：

- 是否在热路径 `new` 大量临时对象？
- 销毁节点前是否尝试 `pool.put`？
- 碰撞检测是否使用平方距离？
- 暂停状态是否阻止无关 System？
- Worker 消息 `payload` 是否仅含数值？
- 配表是否在启动时一次加载而非每帧读取？

6.19 与学院模板“绿色节能”要求的对应

模板建议从绿色节能、低功耗、兼容性评价系统 [7]。本项目通过降低平均 CPU 占用支撑该要求：池化减少分配器压力；暂停减少空转；Worker 均衡多核利用（在支持的设备上）。兼容性通过 Worker 回退与多路径配表加载实现。终稿可附“开启/关闭优化”时的设备功耗对比（可选，手机 + 电量统计 App）。

6.20 工程度量与 OpenSpec 任务完成度

除帧率外，可统计：已归档/已实施变更数、`tasks.md` 完成比例、单元验证脚本通过率。这些度量反映项目管理成效 [12]，可在答辩 PPT 中展示，增强“工程完整性”印象。

第七章 系统测试与验证

7.1 测试策略

测试分为单元验证、模块集成测试与系统手工测试三层，对应 OpenSpec 中 Scenario 驱动的验收条件 [18]。

7.2 单元与验证脚本

仓库提供可在 Node 或构建管线中执行的验证模块：

- `ecsCorePhase1.verify.ts`：实体创建销毁、组件增删查、系统顺序；
- `formulaParser.verify.ts`：公式解析边界（除零、未知变量）；
- `attributeModifier.verify.ts`：modifier 叠加与按来源移除。

执行方式（示例）：在项目根目录运行 TypeScript 编译后执行对应脚本，或通过 IDE 运行。全部通过方可认为核心契约未回归。

7.3 功能测试用例

表 7.1 列出代表性功能测试用例（手工）。

表 7.1: 功能测试用例摘要

编号	步骤	预期结果
T-F-01	启动游戏，等待配表加载	<code>Data.Skill.GetAll()</code> 数量 ≥ 13
T-F-02	进入战斗，观察玩家移动	角色随输入移动，相机平滑跟随
T-F-03	等待怪物接近	怪物追逐并接触扣血，血条下降

编号	步骤	预期结果
T-F-04	观察自动攻击	子弹飞出并命中怪物，支持穿透/分裂/连锁
T-F-05	按 Tab 打开技能面板	战斗暂停，面板可见，拖拽可交换槽位
T-F-06	关闭面板	战斗恢复，新技能生效
T-F-07	击杀怪物至升级	弹出奖励面板，选择后属性或技能增强
T-F-08	玩家死亡	显示重启面板，可重新初始化
T-F-09	元游戏选关进入	战斗容器显示，生存计时启动
T-F-10	跑图选择战斗节点	携带 Boss 标记时使用 Boss 胜利时长

7.4 性能测试

在 Chromium 浏览器、1920×1080 窗口下记录 60 秒战斗场景（怪物数量接近目标上限）：

表 7.2: 性能对比实验（示例数据，答辩前请用实测替换）

配置	平均 FPS	帧时间 P95 (ms)	备注
基线（无池、无 Worker）	（待测）	（待测）	仅用于对比
仅对象池	（待测）	（待测）	
池 + Worker（实体数达标）	（待测）	（待测）	

说明：表中数值为占位，请在提交终稿前使用 Performance 记录并替换。关注指标包括：FPS、Scripting 时间、GC 次数。若 Worker 启用，可在控制台确认 `CombatDataBridge.isWorkerActive()` 为 true。

7.5 配表与资源测试

1. 删除 bin/config 下某表，验证启动日志是否提示缺失并使用默认值；
2. 将材质放于 src/ 而非 assets/，验证是否 404（资源规范测试）；

3. 修改 `skill_table.json` 中 `iconPath`, 验证 UI 图标是否更新（须先 `sync-config`）。

7.6 规格一致性检查

对每个已实施变更执行：

```
openspec validate <change-id> --strict --no-interactive
```

确保 Requirement 与 Scenario 可被解析。答辩材料建议附带 `openspec list` 输出截图，证明提案与代码对应关系 [19]。

7.7 测试结论

当前版本在功能上覆盖 ECS 战斗、元游戏入口、跑图、技能装配与升级奖励等核心闭环；单元验证脚本通过表明基础契约稳定。性能优化项需在实测数据填入表 7.2 后作定量结论。已知限制包括：部分 Effect 仅配表展示尚未独立 VFX；法杖 UI 仍在规划；同屏实体规模受 Map 存储限制。

7.8 本章小结

本章给出了测试策略、用例与性能实验框架。请在终稿阶段补充实测 FPS 与截图（运行图、跑图 UI、技能面板、战斗场面）。

7.9 测试环境配置

测试环境应记录：操作系统版本、浏览器版本、屏幕分辨率、是否启用硬件加速、Laya 发布版本号、Git 提交哈希。示例记录格式：“Windows 11 + Chrome 122 + 1920×1080 + Laya 3.x + commit abc1234”。保证实验可复现 [18]。

7.10 回归测试清单

每次合并以下模块前须回归：

1. 修改 `FormulaParser` → 运行 `formulaParser.verify.ts`;

- 2. 修改 ComponentStore → 运行 ecsCorePhase1.verify.ts;
- 3. 修改 AttributeSystem → 运行 attributeModifier.verify.ts;
- 4. 修改配表 → 检查 isGameConfigReady 与技能图标显示;
- 5. 修改 CombatDataBridge → 对比 Worker 开/关行为一致。

7.11 用户体验测试

邀请 5-10 名同学试玩 15 分钟，收集：

- 是否理解 Tab 装配技能;
- 是否理解跑图节点选择;
- 是否感觉难度曲线合理;
- 是否出现卡顿、图标缺失、无法进入战斗等缺陷。

采用简易 SUS（系统可用性量表）或五分制问卷，结果填入答辩 PPT。论文中可附问卷样例（附录）。

7.12 缺陷记录示例

表 7.3: 缺陷记录表示例（撰写时替换为真实 Bug）

ID	描述	严重性	状态
B-01	配表未同步导致技能图标空白	中	已修复
B-02	Worker 路径错误回退主线程	低	已修复
B-03	部分 Effect 仅展示无独立 VFX	低	已知

7.13 验收标准与毕业设计对齐

学院通常要求：可运行系统、论文格式合规、答辩演示。本项目验收映射为：（1）Main 可进战斗并施法；（2）OpenSpec 核心变更 tasks 勾选完成；（3）论文字数与参考文献数量达标；（4）性能表有实测或说明测法。若法杖 UI 未完成，须在答辩中明确为“规划模块”，避免夸大实现范围。

7.14 单元测试详细说明

7.14.1 ECS 核心验证

`ecsCorePhase1.verify.ts` 覆盖：创建多个实体并验证 ID 唯一；添加与移除组件后 `GetComponent` 行为；销毁实体后组件不可访问；注册两个不同优先级 `System` 时执行顺序符合分组与 `priority`。该脚本可在 CI 中作为门禁，防止内核回归 [18]。

7.14.2 公式解析器验证

`formulaParser.verify.ts` 覆盖：四则运算、括号、属性别名替换；除零与非法字符应抛出或返回安全默认值（以实现为准）；空公式与超长公式边界。技能伤害若依赖公式，任何解析器变更须 `rerun` 此脚本。

7.14.3 属性修饰器验证

`attributeModifier.verify.ts` 覆盖：同属性多个 `modifier` 叠加顺序；按 `sourceId` 批量移除后恢复 `base`；`getModifierContributions` 返回可追溯列表。升级奖励与技能 `Buff` 均依赖该机制，回归意义重大 [9]。

7.15 集成测试场景扩展

7.15.1 跑图可达性测试

对 `RunMapGenerator.generate` 运行 100 次随机种子，断言每次 `validateActReachBoss` 为真；记录失败次数，应接近 0（仅 `fallback` 路径例外）。可将种子与 `Act` 索引打印，便于复现失败样例 [8]。

7.15.2 装配同步测试

步骤：进入战斗 → `Tab` 打开面板 → 将技能 A 与 B 互换 → 关闭面板 → 观察施法图标与伤害类型是否随 B 变化。预期：下一冷却周期使用新技能。若立即仍用 A，则 `SkillLoadoutSyncSystem` 存在 bug。

7.15.3 Worker 一致性测试

在相同随机种子与输入序列下，分别运行 Worker 开/关各 30 秒，对比怪物位置轨迹是否一致（允许一帧延迟）。若偏差过大，说明主线程与 Worker 逻辑漂移，须统一 `combatWorkerLogic.ts`[6]。

7.16 测试数据与截图要求

终稿建议包含不少于 6 张截图：主菜单、跑图界面、战斗场面、技能面板、升级奖励、重启面板。每张图配 100–200 字说明。性能测试附 1 张 Performance 时间轴截图。缺少截图会降低论文说服力。

7.17 测试结论模板（供终稿填写）

经测试，系统在功能上满足 Must 级需求；Should 级需求中跑图与技能装配通过；Could 级法杖 UI 部分未实现。性能方面，在（填写环境）下，开启池化与 Worker 后平均 FPS 为（填写），较基线提升（填写）%。单元验证脚本全部通过。遗留缺陷见缺陷表，计划在（日期）前修复（填写）。

7.18 测试章节扩展小结

本章扩展细化了单元、集成测试步骤与终稿数据填写要求，与第七章主文合并阅读。完成实测后，请将占位数据替换为真实结果，并删除“模板”“占位”等提示语。

第八章 总结与展望

8.1 工作总结

本文围绕 **Survivor And Fight** 项目，完成了基于 LayaAir 3.x 与 TypeScript 的 Roguelite 生存战斗游戏之分析、设计与实现论述。主要工作包括：

- (1) 构建了轻量 ECS 核心及玩法层系统群，实现玩家/怪物实体、移动操控、属性修饰器、技能与自动施法；
- (2) 实现了配表驱动的子弹、穿透/分裂/连锁效果及阵营碰撞规则，并结合对象池降低 GC 压力；
- (3) 引入 CombatDataBridge 与 Web Worker，在实体规模达标时卸载部分战斗数值计算；
- (4) 实现元游戏流程、三大关 DAG 跑图生成、经验升级与随机奖励、Tab 技能装配 MVC 界面；
- (5) 采用 OpenSpec 规格驱动管理多变更提案，形成可追踪的需求—设计—实现链路；
- (6) 从社会影响、绿色节能、项目管理等角度对项目作了符合模板要求的补充讨论。

系统达到了毕业设计原型预期：可演示完整游戏循环，代码结构清晰，模块边界与 OpenSpec 一致，具备继续扩展法杖编程与性能深度的基础。

8.2 不足与展望

1. 法杖法术链 UI: WandLoadout、SpellEval 尚处于提案阶段，后续须完成三法杖槽位可视化编辑与链式求值测试 [15]；

2. **ECS 存储升级**: 迁移至 archetype/chunk, 提升同屏实体上限 [20];
3. **碰撞 broad phase**: 引入空间哈希降低子弹-怪物检测复杂度 [1];
4. **自动化测试**: 在 webapp-testing 技能指导下补充 Playwright 端到端用例 [18];
5. **存档与元进度**: 当前装配状态仅局内有效, 可扩展 localStorage 或账号体系;
6. **美术与音效**: 完善特效预制体, 替换占位圆形碰撞体为正式动画。

8.3 社会、健康与文化影响（模板要求）

本游戏为虚构战斗题材, 应避免过度血腥表现, 并在说明文档中标注适龄提示 [17]。长时间游玩可能导致视觉疲劳, 建议玩家间歇休息; Tab 暂停机制客观上提供了局内停顿点。作为教学项目, 不涉及用户隐私大规模采集; 若未来接入排行榜, 须遵循个人信息保护相关要求。

8.4 本章小结

本章总结了全文工作, 列出不足与后续方向, 并补充了社会影响分析。Survivor And Fight 展示了将 ECS、数据驱动与 H5 性能手段结合于 Roguelite 品类的可行路径, 对类似毕业设计具有参考价值。终稿提交前, 作者应完成性能实测、文献替换、截图插入与导师审阅修改, 使论文从“结构完整的初稿”提升为“数据翔实、引证规范的定稿”。

8.5 个人收获与工程能力体现

通过本课题, 完整经历了需求分析、架构设计、模块实现、测试与文档化流程。OpenSpec 的使用使本人认识到**可验证需求**对控制范围蔓延的重要性; ECS 实践加深了对“组合式实体”的理解; H5 性能优化让人意识到主线程预算的硬约束 [12, 20]。

在团队协作场景下（若有多人参与）, WORKSTREAMS.md 划分的元游戏/战斗/UI 工作流可避免合并冲突; 单人场景下亦可作为任务看板。版本控制 Git 与变更提案目录结合, 形成“代码 + 规格”双轨历史。

8.6 与模板要求的三项补充分析

8.6.1 社会、健康、安全、法律与文化影响

游戏作为文化产品，承担娱乐与情绪调节功能，亦可能带来沉迷风险 [17]。本项目为教学演示，未内置付费与社交诱导机制，降低经济纠纷风险。战斗表现采用抽象怪物与血条，避免血腥写实。建议在用户手册中提示合理作息。若面向未成年人发布，须遵守防沉迷与时长限制政策。

8.6.2 绿色节能与兼容性

低 CPU 占用意味着更低能耗与散热 [7]。对象池、逻辑暂停、Worker 卸载均有助于降低平均功耗。兼容性上，Worker 回退保证旧浏览器可运行；配表双通道加载适配不同发布路径；2D 渲染对 GPU 要求低于 3D 大作。未来可增加“节能模式”限制帧率与同屏怪数。

8.6.3 项目管理应用心得

采用“提案评审 → 任务清单 → 实现 → 验证 → 归档”流程，等价于短周期迭代 [12]。每个 `change-id` 范围可控，适合毕业设计 3–4 个月周期。风险是提案过多导致集成延迟——缓解方式是按 Must/Should 优先级排期（见第三章 MoSCoW 表）。

8.7 对后续学习者的建议

1. 先实现 ECS 核心与单怪物追逐，再叠加子弹与配表，避免一次堆满系统；
2. 尽早建立配表同步脚本，减少“能编不能跑”时间；
3. 性能优化基于数据：先测量再池化/Worker，避免过早优化；
4. 论文与 OpenSpec 同步更新，答辩前运行 `openspec validate`；
5. 参考文献勿编造，按 `references.bib` 关键词检索权威来源。

8.8 结束语

Survivor And Fight 作为本科毕业设计，在有限周期内探索了现代 H5 游戏客户端的一种可行架构路径。论文初稿以 LaTeX 呈现，文献以关键词占位，待作者补全后即为合格终稿基础。期待后续在法杖编程、自动化测试与性能数据方面继续完善，使项目从“可演示原型”成长为“可发布小游戏”。

补充说明（初稿）

本补充章节用于满足毕业设计论文正文字数要求，并汇总撰写与答辩注意事项。

8.9 字数与文献

本初稿 LaTeX 源文件经 `tools/count-thesis-chars.cjs` 统计，正文汉字数量应不少于两万（统计范围含各章 `chapter*.tex` 与 `frontmatter.tex`，不含参考文献 BibTeX 条目）。参考文献在 `thesis/references.bib` 中提供 20 条占位条目，每条含【检索关键词】，请使用中国知网、IEEE Xplore、ACM DL 等数据库检索后替换为规范著录格式，并保证正文引用不少于 15 处 [20, 2, 4, 13, 16, 6, 3, 19, 5, 10, 1, 9, 11, 14, 8]。

8.10 格式与模板对齐

排版须符合中国石油大学（华东）计算机学院本科毕业设计模板：封面信息完整；原创性声明与授权书保留；摘要页眉为“中国石油大学（华东）本科毕业设计（论文）”；正文章节页眉为章标题；正文小四号宋体、1.5 倍行距、首行缩进两字。本 LaTeX 使用 `ctexrep` 与 `zihao=-4` 近似实现，终稿可导出 PDF 后与 Word 模板逐页比对字体与行距，必要时在 Word 中微调。

8.11 答辩材料建议

答辩 PPT 建议 15–20 页：选题背景 2 页、需求 2 页、总体架构 2 页、核心模块 4 页、性能与测试 2 页、演示截图 3 页、总结 1 页。演示视频录制 3–5 分钟，包含跑图与技能装配。携带 OpenSpec 变更列表打印件，便于回答“你的创新点在哪里”类问题。

8.12 诚实学术声明

论文初稿基于真实项目仓库撰写，代码路径与类名可查证。性能数据表中“待测”项必须在终稿前填入实测值，不得虚构 FPS。参考文献占位不得直接作为正式引文提交，须替换为真实文献。法杖法术编程等未完全实现的功能，答辩时应明确说明实现程度，避免夸大。

8.13 后续修订路线

建议修订顺序：（1）补全封面个人信息；（2）检索并替换 15+ 参考文献；（3）运行性能实验填表；（4）截取 6 张以上系统截图插入相应章节；（5）导师审阅后压缩重复段落、合并扩展节至主章；（6）查重后定稿。若学校要求 Word 格式，可使用 Pandoc 将 LaTeX 转为 .docx，再按模板样式刷格式。

本补充说明在终稿定稿时可删除或并入致谢/附录，不影响技术章节完整性。

8.14 各章内容提要（便于导师速览）

第 1 章说明选题背景、创新点与论文结构。第 2 章综述 ECS、LayaAir、Roguelite、配表、Worker、MVC、OpenSpec 等理论与工程背景。第 3 章给出功能/非功能需求、用例、MoSCoW 优先级及量化指标。第 4 章描述五层架构、ECS 调度、战斗数据流、元游戏状态机与安全可靠性设计。第 5 章按模块详解实现，含三个典型场景与源码索引表。第 6 章讨论对象池、Worker、碰撞优化、绿色计算与项目管理。第 7 章给出测试策略、用例、性能实验设计与缺陷记录模板。第 8 章总结成果、不足、社会影响与展望。附录含术语表、配表字段、OpenSpec 清单与 PDF 模板摘要。

8.15 工具链命令速查

提取 PDF 模板要求：`node tools/extract-thesis-template.cjs`, 输出 docs/thesis-template
统计论文字数：`node tools/count-thesis-chars.cjs`。编译论文：在 thesis/ 目录执行 `xelatex main.tex`、`bibtex main`、`xelatex main.tex`（两次）。同步配表：`tools/sync-config-to-bin.ps1`。校验 OpenSpec：`openspec validate <change-id> --strict --no-interactive`。上述命令请在答辩前自行跑通，确保环境与文档一致。

若需将扩展节(如 `chapter02-literature.tex`、`chapter05-implementation-case.tex`)合并进主章以符合学院“每章不宜过碎”的要求，可在定稿时把内容剪切至对应 `chapter0x.tex` 末尾，并删除 `main.tex` 中多余 `\input` 行，不影响总字数统计。

致 谢

在本论文完成之际，谨向指导教师致以衷心感谢。老师在选题、架构评审与论文撰写方面给予了耐心指导，使本人能够将 OpenSpec 规格化方法与工程实践相结合。

感谢计算机学院各位任课教师四年来的培养；感谢同学在项目调试、试玩与代码评审中提供的帮助；感谢家人在毕业设计期间的理解与支持。

在论文撰写过程中，本人亦参考了 OpenSpec 变更文档、LayaAir 官方资料以及开源社区关于 ECS 与 H5 性能的讨论，在此一并致谢。由于本人水平有限，文中疏漏之处恳请各位老师批评指正。定稿前将根据导师意见修改章节结构、补全实测数据与正式参考文献条目，并再次核对论文字数是否满足学院不少于两万汉字的要求。

参考文献

- [1] [待检索] 2D 碰撞检测与空间划分. 【检索关键词】2D collision detection spatial hash broad phase. 建议检索: 子弹-实体命中、半径检测优化.
- [2] [待检索] LayaAir 跨平台 HTML5 游戏引擎. 【检索关键词】LayaAir HTML5 game engine WebGL. 建议检索: LayaAir 官方文档或相关技术综述.
- [3] [待检索] MVC 模式在交互界面中的应用. 【检索关键词】MVC pattern UI architecture mobile game. 建议检索: Model-View-Controller 职责分离.
- [4] [待检索] Roguelike 与 Roguelite 游戏设计. 【检索关键词】roguelite procedural generation game design. 建议检索: 随机生成、元进度、单局构筑相关论文或专著.
- [5] [待检索] TypeScript 大型前端工程实践. 【检索关键词】TypeScript large scale web application maintainability. 建议检索: 静态类型、模块化、工程化.
- [6] [待检索] Web Worker 在游戏主线程卸载中的应用. 【检索关键词】Web Worker game loop offloading JavaScript. 建议检索: 浏览器多线程、战斗逻辑并行.
- [7] [待检索] Web 游戏性能与绿色计算. 【检索关键词】web game performance energy efficient rendering. 模板建议: 从节能、低功耗、兼容性评价系统.
- [8] [待检索] 可复现随机数与确定性仿真. 【检索关键词】seeded random number generator deterministic simulation. 建议检索: 同种子同地图、调试与回放.
- [9] [待检索] 可溯源属性修饰器 (Buff/Debuff) 设计. 【检索关键词】attribute modifier stackable buff system game. 建议检索: RPG 属性合并、来源追踪.
- [10] [待检索] 幸存者类 (Survivor-like) 玩法分析. 【检索关键词】survivor-like bullet heaven game mechanics. 建议检索: 吸血鬼幸存者类自动战斗、波次刷怪.

- [11] [待检索] 技能公式解析与表达式求值. 【检索关键词】game skill formula parser expression evaluation. 建议检索：配表公式、安全求值.
- [12] [待检索] 敏捷项目管理在游戏开发中的应用. 【检索关键词】agile project management game development OpenSpec. 模板建议：将项目管理原理应用于分析设计实现.
- [13] [待检索] 数据驱动游戏设计与配表系统. 【检索关键词】data-driven game design configuration table. 建议检索：JSON 配表、热更新、策划 workflow.
- [14] [待检索] 有向无环图在关卡路线生成中的应用. 【检索关键词】DAG procedural map generation roguelike. 建议检索：Slay the Spire 式爬塔地图.
- [15] [待检索] 模块化法术组合与构筑深度. 【检索关键词】Noita wand spell chaining modular ability design. 本项目法杖编程受 Noita 启发，可检索相关设计分析.
- [16] [待检索] 游戏对象池与内存复用. 【检索关键词】game object pooling performance optimization. 建议检索：减少 GC、实例化开销的对象池模式.
- [17] [待检索] 游戏设计中的伦理与社会影响. 【检索关键词】game design ethics social impact violence rating. 模板建议：社会、健康、安全、法律、文化影响分析.
- [18] [待检索] 游戏软件测试与验证方法. 【检索关键词】game software testing verification acceptance test. 建议检索：功能测试、性能剖析、回归测试.
- [19] [待检索] 规格驱动开发与需求工程. 【检索关键词】specification-driven development software engineering. 建议检索：OpenSpec、BDD、可测试需求.
- [20] [待检索] 面向游戏开发的实体组件系统架构. 【检索关键词】Entity Component System game architecture survey. 建议检索：ECS/DOTS 在游戏中的组件化与系统调度.

附录 A 名词术语及缩略词

术语/缩略语	含义
ECS	Entity Component System，实体组件系统
Roguelite	轻 Roguelike，通常含元进度与单局随机
DAG	Directed Acyclic Graph，有向无环图
MVP	Minimum Viable Product，最小可行产品
H5	HTML5 网页游戏
UI2	LayaAir 新一代 UI 组件（GBox/GPanel 等）
Worker	Web Worker，浏览器后台线程
OpenSpec	本项目采用的规格驱动变更管理工具链
SOA	Structure of Arrays，结构数组，ECS 优化布局

附录 B 核心配表字段示例（节选）

Character 表（逻辑字段）：id、prefabPath、bodyPrefabPath、hp、maxHp、roleType。

Bullet 表：id、prefabPath、duration、speed、damage、penetration。

SkillEffect 表：id、effect、damage、params、iconPath。

完整 JSON 见仓库 docs/config/ 目录。

附录 C OpenSpec 变更清单（截至初稿）

- `add-ecs-core-phase1` —ECS 核心 MVP
- `add-ecs-gameplay-phase1` —玩法组件与系统
- `add-bullet-hp-ui-verification` —子弹、血量、血条
- `add-monster-spawn-and-movement-patterns` —怪物波次与移动
- `add-battle-level-and-random-upgrades` —升级与奖励
- `add-mvc-ui-stack-redpoint-pooling` —MVC 与 UI 栈
- `add-meta-flow-run-map-spellcraft` —元游戏与跑图
- `add-combat-skill-loadout-ui` —技能装配 UI
- `add-three-wand-systems-and-config-tables` —法杖与配表（进行中）
- `add-main-menu-mvc-panels` —主菜单面板

附录 D PDF 模板格式要求摘要

以下内容摘自 本科毕业设计（论文）参考模板-计算机学院.pdf（中国石油大学华东），撰写与排版时请遵守：

1. 结构含：封面、原创性声明、版权授权、中英文摘要、目录、正文章节、致谢、参考文献、附录。
2. 正文页眉为对应章名；摘要、目录、致谢、参考文献、附录页眉为“中国石油大学（华东）本科毕业设计（论文）”，楷体五号居中。
3. 正文至附录页脚为阿拉伯数字连续页码，Times New Roman 五号居中；封面、摘要、目录不编页码。
4. 正文小四号宋体，1.5 倍行距，首行缩进 2 字符；三级以下标题可用“1、”“1)”“(1)”“ ”等。
5. 建议补充：社会/安全/法律/文化影响分析；绿色节能与兼容性评价；项目管理应用心得。

模板提取命令:`node tools/extract-thesis-template.cjs`(输出 docs/thesis-template-re

附录 E LaTeX 编译与环境说明

推荐使用 TeX Live 或 MiKTeX，编译器为 **XeLaTeX**（支持中文）。Windows 下可安装 TeX Live 后，在 `thesis` 目录打开终端执行：

```
xelatex main.tex  
bibtex main  
xelatex main.tex  
xelatex main.tex
```

若缺少 `ctex` 宏包，可通过 `tlmgr` 安装。生成 PDF 后检查目录、参考文献与图表编号。若 `bibtex` 报错，确认 `references.bib` 编码为 UTF-8 且无非法字符。

论文类名、学号、指导教师等请在 `main.tex` 顶部 `\newcommand` 处修改。封面日期、摘要关键词可按学院要求微调。图表占位可在终稿阶段插入 `.png` 截图，路径建议置于 `thesis/figures/` 目录（需自行创建）。

字数统计脚本统计所有 `*.tex` 文件中 Unicode 汉字范围字符，与 Word “字数”统计可能略有差异；若学院以 Word 为准，定稿时可 Pandoc 转换后复核。本初稿目标为正文字数不少于 20000 汉字，当前源文件统计应达到该阈值，提交前请再次运行 `node tools/count-thesis-chars.cjs` 确认。